

MAS2901: Statistical Inference

Matthew Fisher

Table of contents

Preface	3
What is Statistical Inference?	3
Further Reading	4
I Part I — Foundations	5
1 Probability Theory	6
1.1 Probability and Events	6
1.2 Random Variables	13
1.2.1 Joint, Marginal, and Conditional Densities	14
1.3 Expectations, Variances and Covariances	14
2 What is Statistical Inference?	15
2.1 Statistical Inference as Inverse Probability Theory	16
2.1.1 Statistical Inference by Eye	16
2.2 Statistical Models	19
2.3 Exercises	22
3 Types of Inference	23
3.1 Philosophies of Inference	23
3.1.1 Frequentist Inference	23
3.1.2 Bayesian Inference	23
3.1.3 Key Differences	24
3.2 Inference Tasks	24
II Part II — Parameter Estimation	26
4 Frequentist Point Estimation	27
4.1 Estimators vs. Estimates	27
4.1.1 Sample Mean and Variance	28
4.1.2 The Randomness of Estimators	28
4.2 Bias, Variance, MSE and Consistency	37
4.2.1 Examples	38
4.3 Central Limit Theorem (CLT)	40

5	Likelihood	41
5.1	From Joint Density to Likelihood	41
5.2	Likelihood Function	42
5.2.1	Computing Likelihood Functions	54
5.3	Maximum Likelihood Estimation	55
5.3.1	Computing MLEs	55
5.4	Summary	56
6	Bayesian Inference	57
6.1	Introduction to the Bayesian approach	57
6.1.1	Why is this useful?	57
6.1.2	Python	58
6.1.3	R	59
6.2	Bayes' Theorem	60
6.3	Bayes' Theorem in Statistics	61
6.3.1	Python	66
6.3.2	R	67
6.4	Bayesian vs. Frequentist (again)	68
III	Part III - Interval Estimation	69
7	Frequentist Interval Estimation	70
7.1	The Frequentist Perspective	70
7.2	One Observed Sample	70
7.3	Deriving Confidence Intervals	71
7.3.1	The Normal case (Known Variance)	71
7.3.2	Normal Case (Unknown Variance)	72
7.3.3	Approximate Intervals via the CLT	72
8	Bayesian Intervals	73
8.1	Bayesian Credible Intervals	73
IV	Part IV — Hypothesis Testing	74
9	Frequentist Hypothesis Testing	75
9.1	Formal Setup	75
9.2	Test Statistics and the Testing Procedure	77
9.3	Types of Errors	78
10	Bayes Factors	79

V Additional Material	80
Formula Sheet	81
Discrete distributions	81
Bernoulli	81
Binomial	81
Poisson	81
Geometric	82
Negative Binomial	82
Continuous distributions	82
Uniform (continuous)	82
Exponential	83
Gamma	83
Normal	83
Lognormal	83
Chi-squared	84
Student- t	84
F distribution	84
Beta	84
Multivariate Normal	85
Integral reminder	85
Revision Sheet	86
Distribution Explorer	87
10.1 Bernoulli	88
10.2 Binomial	89
10.3 Poisson	90
10.4 Geometric	91
10.5 Negative Binomial	92
10.6 Uniform	93
10.7 Exponential	94
10.8 Gamma	94
10.9 Normal	95
10.10 Lognormal	96
10.11 Chi-squared	97
10.12 Student t	98
Question Bank	99

Preface

Welcome to the online notes for **MAS2901 — Statistical Inference**.

This 10-credit module at Newcastle University introduces the two main approaches to *statistical inference*:

- **Frequentist inference**
- **Bayesian inference**

What is Statistical Inference?

Statistical inference is the process of using data from a *sample* to learn about a *population*. Sometimes the population is concrete (e.g. “all people in the country today”). In other situations we want to generalise beyond those we sampled to a broader *target population* (sometimes called a *superpopulation*), such as “future patients with similar characteristics”. Clear definitions and sensible assumptions are essential for valid conclusions.

There are two main types of statistical inference:

- **Frequentist inference.** Parameters are treated as **fixed but unknown**. Probability statements are about *data we might observe* under repeated sampling from the same model.
- **Bayesian inference.** Parameters are treated as **random** with a *prior distribution*. Probability statements are about the *parameter given the data* via the posterior distribution.

The examples below are only to build intuition about the core difference between frequentist and Bayesian inference. Don’t worry about the details—we’ll cover them carefully later in the course.

Example 0.1 (A comparison of Frequentist vs Bayesian inference). A survey asks $n = 20$ students if they prefer study space A or B .

Suppose $x = 12$ prefer A . Compare a frequentist and a Bayesian estimate of the long-run proportion p who would choose A .

💡 Solution

Frequentist. The natural estimator is the sample proportion

$$\hat{p} = \frac{x}{n} = \frac{12}{20} = 0.60.$$

Bayesian (with a uniform prior). Take a $\text{Beta}(1, 1)$ prior for p . The posterior is

$$p \mid x \sim \text{Beta}(1 + x, 1 + n - x) = \text{Beta}(13, 9),$$

whose mean is

$$\mathbb{E}[p \mid x] = \frac{13}{13 + 9} = \frac{13}{22} \approx 0.591.$$

Interpretation. The frequentist estimate $\hat{p}$ is a function of the data and treats p as fixed.

The Bayesian estimate is an expectation *of p itself* under the posterior distribution.

Example 0.2 (Interpreting probability statements). Decide whether each statement is *frequentist* or *Bayesian* in spirit.

1. “If we repeated the survey many times, the method we use would produce intervals that capture p about 95% of the time.”
2. “Given the data from this survey and our prior, there is a 95% probability that p lies between 0.45 and 0.75.”

💡 Solution

1. **Frequentist.** This describes *long-run coverage* of a procedure over repeated samples with p fixed.
2. **Bayesian.** This is a probability statement *about p given the observed data* (a credible interval).

Further Reading

- asdasdasds

Part I

Part I — Foundations

1 Probability Theory

Learning objectives

- Define and work with events (union, intersection, complement) and apply inclusion–exclusion.
- Distinguish mutually exclusive, exhaustive, and independent events.
- Compute conditional probabilities and apply Bayes’ theorem.
- Use partitions and the law of total probability.
- Define random variables and their distribution functions: cdf, pmf (discrete), and pdf (continuous).
- Compute probabilities from pmfs/pdfs/cdfs (using sums or areas) and check normalisation.

Before we delve into statistical inference, we briefly review key ideas from probability theory.

Note on this section

- This section is provided **for review and reference only**.
- It will not be examined in detail, but the ideas are useful background for later topics.
- Much of the content should already be familiar.

1.1 Probability and Events

When working with probability we need three basic ingredients¹

1. The **sample space** Ω . all possible outcomes of an experiment.
Example: for a coin toss, $\Omega = \{\text{Heads}, \text{Tails}\}$.
2. **Events** subsets of the sample space. We use capital letters $A, B, \dots$ to denote events.

¹**Note on Probability Theory.** Formally, a probability space is written as a triple (Ω, Σ, P) , where Σ is the collection of “measurable” events. This level of mathematical detail goes beyond what we need in this course and will **not be assessed**. (See the [Wikipedia article on probability spaces](#) if you are curious.)

- The **complement** of A is A^c , meaning A does *not* occur.
- The **intersection** $A \cap B$ means A *and* B occur.
- The **union** $A \cup B$ means A *or* B (or both) occur.

3. A **probability distribution** $\Pr$. Assigns numbers between 0 and 1 to events, with $\Pr(\Omega) = 1$.

We can write the definitions of complement, intersection and union formally with sets:

Definition 1.1 (Complement, Intersection and Union).

- **Complement:** $A^c = \{x \in \Omega \mid x \notin A\}$.
- **Intersection:** $A \cap B = \{x \in \Omega \mid x \in A \text{ and } x \in B\}$.
- **Union:** $A \cup B = \{x \in \Omega \mid x \in A \text{ or } x \in B\}$.

Visualisation:

```
// Toggle: highlight selection (use explicit values for robust comparisons)
viewof vennOp = Inputs.radio(
  ["None", "A", "B", "A ", "Union", "Intersection"],
  { value: "None", label: "Highlight:" }
)
```

```
//| label: fig-venn
//| fig-cap: "Venn diagram of two events A and B inside the sample space  $\Omega$ ."

venn = {
  const width = 380, height = 280;
  const pad = 18;

  const cxA = 145, cyA = 140, r = 90;    // circle A
  const cxB = 235, cyB = 140;            // circle B

  const svg = d3.create("svg")
```

```

    .attr("width", width)
    .attr("height", height)
    .attr("viewBox", [0, 0, width, height]);

// Sample space  $\Omega$  (bounding rectangle)
const omega = svg.append("rect")
    .attr("x", pad).attr("y", pad)
    .attr("width", width - 2*pad).attr("height", height - 2*pad)
    .attr("rx", 10)
    .attr("fill", "none")
    .attr("stroke", "currentColor")
    .attr("stroke-opacity", 0.5);

// Label  $\Omega$ 
svg.append("text")
    .attr("x", pad + 8).attr("y", pad + 16)
    .text(" $\Omega$ ")
    .style("font-weight", 400)
    .style("font-size", "14px");

// Colors (CSS vars with fallbacks)
const colA = "var(--brand-teal, #14b8a6)";
const colB = "var(--brand-red, #ef4444)";
const hilite = "var(--brand-orange, #F39C65)";
const grid = "var(--plot-grid, #94a3b8)";

// --- Defs: clips & mask for  $A^c$  ---
const defs = svg.append("defs");

defs.append("clipPath").attr("id", "clipA")
    .append("circle").attr("cx", cxA).attr("cy", cyA).attr("r", r);

defs.append("clipPath").attr("id", "clipB")
    .append("circle").attr("cx", cxB).attr("cy", cyB).attr("r", r);

// Mask for  $A^c$ : white keeps, black removes (subtract circle A from  $\Omega$  rect)
const m = defs.append("mask").attr("id", "maskAc");
m.append("rect")
    .attr("x", pad).attr("y", pad)
    .attr("width", width - 2*pad).attr("height", height - 2*pad)
    .attr("rx", 10)
    .attr("fill", "white");

```

```

m.append("circle")
  .attr("cx", cxA).attr("cy", cyA).attr("r", r)
  .attr("fill", "black");

// Base sets (always lightly transparent)
svg.append("circle")
  .attr("cx", cxA).attr("cy", cyA).attr("r", r)
  .attr("fill", colA).attr("fill-opacity", 0.18);

svg.append("circle")
  .attr("cx", cxB).attr("cy", cyB).attr("r", r)
  .attr("fill", colB).attr("fill-opacity", 0.18);

// Highlight layer
if (vennOp === "A") {
  svg.append("circle")
    .attr("cx", cxA).attr("cy", cyA).attr("r", r)
    .attr("fill", hilite).attr("fill-opacity", 0.99);

} else if (vennOp === "B") {
  svg.append("circle")
    .attr("cx", cxB).attr("cy", cyB).attr("r", r)
    .attr("fill", hilite).attr("fill-opacity", 0.99);

} else if (vennOp === "A ") {
  // Draw  $\Omega$  in orange, but masked to remove A
  svg.append("rect")
    .attr("x", pad).attr("y", pad)
    .attr("width", width - 2*pad).attr("height", height - 2*pad)
    .attr("rx", 10)
    .attr("fill", hilite).attr("fill-opacity", 0.99)
    .attr("mask", "url(#maskAc)");

} else if (vennOp === "Union") {
  // A B: overlay both circles solid orange
  svg.append("circle")
    .attr("cx", cxA).attr("cy", cyA).attr("r", r)
    .attr("fill", hilite).attr("fill-opacity", 0.99);
  svg.append("circle")
    .attr("cx", cxB).attr("cy", cyB).attr("r", r)
    .attr("fill", hilite).attr("fill-opacity", 0.99);
}

```

```

} else if (vennOp === "Intersection") {
  // A ∩ B: clip B by A (equivalently clip A by B)
  svg.append("g").attr("clip-path", "url(#clipA)")
    .append("circle")
    .attr("cx", cxB).attr("cy", cyB).attr("r", r)
    .attr("fill", hilite).attr("fill-opacity", 0.99);
}

// Labels A and B
svg.append("text")
  .attr("x", cxA - 26).attr("y", cyA - 8)
  .text("A")
  .style("font-size", "14px")
  .style("font-weight", 600);

svg.append("text")
  .attr("x", cxB + 22).attr("y", cyB - 8)
  .text("B")
  .style("font-size", "14px")
  .style("font-weight", 600);

// Subtle baseline
svg.append("line")
  .attr("x1", pad).attr("x2", width - pad)
  .attr("y1", height - pad - 0.5).attr("y2", height - pad - 0.5)
  .attr("stroke", grid).attr("stroke-opacity", 0.2);

return svg.node();
}

```

Definition 1.2 (Addition rule (inclusion–exclusion)).

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B).$$

Definition 1.3 (Mutually exclusive events). Two events A and B are **mutually exclusive** if they cannot both occur:

$$\Pr(A \cap B) = 0.$$

Definition 1.4 (Exhaustive events). Two events A and B are **exhaustive** if at least one must occur:

$$\Pr(A \cup B) = 1.$$

(More generally, a collection of events is exhaustive if their union equals the whole sample space.)

Definition 1.5 (Independence). Two events A and B are **independent** if knowing that one occurs does not change the probability of the other:

$$\Pr(A \cap B) = \Pr(A) \Pr(B).$$

Definition 1.6 (Conditional probability). If $\Pr(B) > 0$, the probability of A given B is

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

Theorem 1.1 (Bayes' Theorem). *If $\Pr(B) > 0$, then*

$$\Pr(A \mid B) = \frac{\Pr(B \mid A) \Pr(A)}{\Pr(B)}.$$

Proof. From the definition of conditional probability [1.6](#),

$$\Pr(A \mid B) = \frac{\Pr(A \cap B)}{\Pr(B)}.$$

But the intersection $A \cap B$ can also be written the other way round:

$$\Pr(A \cap B) = \Pr(B \cap A).$$

Using the definition of conditional probability again,

$$\Pr(B \cap A) = \Pr(B \mid A) \Pr(A).$$

Substituting this into the first expression gives

$$\Pr(A \mid B) = \frac{\Pr(B \mid A) \Pr(A)}{\Pr(B)}.$$

This completes the proof.

□

Theorem 1.2 (Law of Total Probability). *If $B_1, \dots, B_n$ form a **partition** of the sample space (mutually exclusive and exhaustive events), then for any event A :*

$$\Pr(A) = \sum_{i=1}^n \Pr(A \mid B_i) \Pr(B_i).$$

Example 1.1 (A quick inclusion–exclusion check). Roll a fair six-sided die. Let $A = \{\text{even}\}$ and $B = \{\text{number} \geq 4\}$. Compute $\Pr(A \cup B)$.

Solution 1.1. $\Pr(A) = 3/6 = 1/2$ (even outcomes 2, 4, 6).

$\Pr(B) = 3/6 = 1/2$ (outcomes 4, 5, 6).

$\Pr(A \cap B) = 2/6 = 1/3$ (outcomes 4, 6).

By inclusion–exclusion,

$$\Pr(A \cup B) = \frac{1}{2} + \frac{1}{2} - \frac{1}{3} = \frac{2}{3}.$$

Example 1.2 (Student Example). Seventy percent of students are attentive in lectures and thirty percent are not. If a student is attentive the probability of passing the course is 0.8. If a student is not attentive the probability of passing the course is 0.1.

1. A student is selected at random. What is the probability that they pass the course?
2. Given that the student passed the course, what is the probability that they were attentive?

Solution 1.2. Let

- P denote the event of a student passing the course.
- A denote the event of a student being attentive.
- A^c denote the event of a student not being attentive.

1. Attentive and inattentive form a partition. By the law of total probability,

$$\Pr(P) = \Pr(P \mid A) \Pr(A) + \Pr(P \mid A^c) \Pr(A^c) = 0.8 \times 0.7 + 0.1 \times 0.3 = 0.59.$$

2. Using Bayes' theorem,

$$\Pr(A \mid P) = \frac{\Pr(P \mid A) \Pr(A)}{\Pr(P)} = \frac{0.8 \times 0.7}{0.59} = 0.949 \text{ (3 s.f.)}.$$

1.2 Random Variables

We use upper-case letters for random variables, e.g. $X, Y, X_1, X_2, \dots$.

- **Cumulative distribution function (cdf).** For any random variable X ,

$$F(x) = \Pr(X \leq x).$$

- **Discrete case (pmf).** If X is discrete, its *probability mass function* is

$$p(x) = \Pr(X = x), \quad \sum_x p(x) = 1.$$

- **Continuous case (pdf).** If X is continuous, its *probability density function* satisfies

$$f(x) = \frac{dF(x)}{dx}, \quad \Pr(a < X \leq b) = \int_a^b f(x) dx, \quad \int_{-\infty}^{\infty} f(x) dx = 1.$$

Example 1.3 (Discrete CDF.). Let X be the outcome of a fair six-sided die. Find $p(x)$ and $F(3)$.

Solution 1.3. $p(x) = \Pr(X = x) = 1/6$ for $x \in \{1, 2, 3, 4, 5, 6\}$, and 0 otherwise.
 $F(3) = \Pr(X \leq 3) = \Pr(\{1, 2, 3\}) = 3 \times (1/6) = 1/2$.

Example 1.4. Let $X \sim \text{Uniform}(0, 1)$. Compute $\Pr(0.2 < X < 0.5)$.

Solution 1.4. Here $f(x) = 1$ for $0 < x < 1$ and 0 otherwise.

$$\Pr(0.2 < X < 0.5) = \int_{0.2}^{0.5} 1 dx = 0.3.$$

1.2.1 Joint, Marginal, and Conditional Densities

Suppose X and Y are random variables with joint PDF/PMF $f_{XY}(x, y)$ (often written $f(x, y)$). From this joint distribution we can define:

- **Marginal distributions:** the distribution of each variable on its own.
- **Conditional distributions:** the distribution of one variable given the other.
- **Independence:** the case where the joint factorises.

Definition 1.7 (Marginal Distributions). If we only care about X , regardless of Y , we integrate/sum out Y :

$$f_X(x) = \int f(x, y) \, dy.$$

Similarly for $f_Y(y)$.

Note: knowing the marginals does **not** determine the joint.

Definition 1.8 (Conditional Densities). For y in the support of Y with $f_Y(y) > 0$, we define:

$$f_{X|Y}(x | y) = \frac{f(x, y)}{f_Y(y)}.$$

This describes the distribution of X *given* $Y = y$.

Theorem 1.3 (Law of Total Probability). *We can recover the marginal by “averaging” the conditional over Y :*

$$f_X(x) = \int f_{X|Y}(x | y) f_Y(y) \, dy.$$

Definition 1.9 (Independence). X and Y are independent if and only if

$$f(x, y) = f_X(x) f_Y(y).$$

In this case, $f_{X|Y}(x | y) = f_X(x)$: knowing Y tells us nothing about X .

1.3 Expectations, Variances and Covariances

2 What is Statistical Inference?

Learning objectives

- Define *population* and *sample*, and identify each in simple scenarios.
- State the *goal* of statistical inference: using a sample to learn about a population distribution.
- Recognise the idea of a *random sample* $X_1, \dots, X_n$ and the iid assumption.
- Define a *statistical model* as a set of distributions $\{P_\theta : \theta \in \Theta\}$.
- Identify the *parameter(s)* θ and the *parameter space* Θ for a given model.
- Write down a suitable statistical model for a short verbal problem.

Statistical inference underpins experimental science, machine learning, clinical trials, and more. The overall aim is:

- Given a **sample** (the data you have) from a **population** (the broader group you care about),
- Draw a justified **conclusion about the population's distribution**.

Typical conclusions include estimating a proportion or mean, quantifying uncertainty, comparing groups, or predicting future outcomes.

Common misconceptions and how to avoid them

- **Parameter $\neq$ statistic.** A *parameter* θ is a property of the population/model (e.g. p , μ , σ^2). A *statistic* (e.g. $\bar{X}$, a sample proportion) is computed from the sample to learn about θ .
- **“Random sample” $\neq$ “whatever data we found”.** The iid assumption is part of the design; convenience samples or dependence can bias inference.
- **Models are approximations.** A model is a useful idealisation, not the truth. Choose $\mathcal{P}$ sensibly, then check fit later and revise if needed.

Examples: 1. Simple one-dimensional example 2. Slightly more convoluted two-dimensional example 3. Machine learning (images of cats)

2.1 Statistical Inference as Inverse Probability Theory

We assume the population can be represented by a probability distribution $f(x \mid \theta)$, where θ is an unknown parameter (or vector of parameters). Our data are usually modelled as an **independent and identically distributed (iid)** sample:

$$X_1, \dots, X_n \stackrel{\text{iid}}{\sim} f(x \mid \theta).$$

- *Independent*: knowing one X_i tells you nothing about another.
- *Identically distributed*: each X_i has the same distribution f .

Inference is the process of learning about θ (and hence about the population distribution) from the observed sample. In practice, always ask whether iid sampling is plausible for your study design.

2.1.1 Statistical Inference by Eye

As an informal starting point, suppose $X_1, \dots, X_n$ are sampled from a $\mathcal{N}(\mu, \sigma^2)$ distribution, with μ and σ^2 unknown. Try to “guess” μ and σ by eye from the data:

```
///| echo: false

html`
<style>
.controls-grid {
  display: grid;
  grid-template-columns: 1fr 1fr 1fr;    /* 3 columns */
  gap: 0.8rem;
  align-items: end;
}
@media (max-width: 800px){
  .controls-grid { grid-template-columns: 1fr; }
}
</style>
`

viewof n = Inputs.range([50, 1000], { value: 200, step: 50, label: "n" })

trueChoices = new Map([
  ["N(0, 1²)",    {mu: 0,  sigma2: 1}],
  ["N(0, 2²)",    {mu: 0,  sigma2: 4}],
```

```

["N(1, 12)", {mu: 1, sigma2: 1}],
["N(-2, 0.52)", {mu: -2, sigma2: 0.25}],
["N(2, 1.52)", {mu: 2, sigma2: 2.25}]
])

viewof trueDist = Inputs.select(trueChoices, {
  label: "True Distribution",
  value: trueChoices.get("N(0, 12)")
})

viewof approxVar = Inputs.range([0.1, 9], { value: 1, step: 0.1, label: "Approx. variance ( 2"
viewof approxMu = Inputs.range([-5, 5], { value: 0, step: 0.1, label: "Approx. mean ( )" })

viewof resample = Inputs.button("Generate new sample")

```

```

//| panel: input
{
  const box = html`<div class="controls-grid"></div>`;
  box.append(
    viewof n,
    viewof trueDist,
    viewof approxMu,
    viewof approxVar,
    viewof resample
  );
  return box;
}

```

```

bodyFont = getComputedStyle(document.body).fontFamily

```

```

// reactive params
mu = trueDist.mu
sigma2 = trueDist.sigma2

// data
data = {
  resample;
  const sigma = Math.sqrt(sigma2);
  const rnd = d3.randomNormal(mu, sigma);

```

```

    return Array.from({length: n}, () => rnd());
  }

  approxSigma = Math.sqrt(approxVar)

  // plot
  {
    const xMin = d3.min(data), xMax = d3.max(data);
    const pad = 0.1 * (xMax - xMin || 1);
    const domain = [xMin - pad, xMax + pad];

    const xs = d3.range(domain[0], domain[1], (domain[1]-domain[0]) / 200);
    const normalPdf = (x, mu, sigma) => (1/(sigma*Math.sqrt(2*Math.PI))) * Math.exp(-0.5*((x-mu)/sigma)**2);
    const curve = xs.map(x => ({x, y: normalPdf(x, approxMu, approxSigma)}));

    const binCount = 30;
    const binWidth = (domain[1] - domain[0]) / binCount;
    const scaledCurve = curve.map(d => ({x: d.x, y: d.y * n * binWidth}));

    return Plot.plot({
      marginLeft: 48,
      marginBottom: 40,
      style: {
        fontSize: 14,
        fontFamily: bodyFont,
      },
      width: 700,
      height: 420,
      x: {label: "x", domain},
      y: {label: "Count"},
      marks: [
        Plot.rectY(data, Plot.binX({y: "count"}, {thresholds: binCount})),
        Plot.line(scaledCurve, {x: "x", y: "y", strokeWidth: 2})
      ]
    });
  }
}

```

(This intuition will be formalised later when we develop estimators and uncertainty quantification.)

2.2 Statistical Models

Whenever we perform statistical inference, we must specify a *statistical model*.

Definition 2.1 (Statistical Model). A **statistical model** is a *set* of candidate population distributions:

$$\mathcal{P} = \{P_\theta \mid \theta \in \Theta\},$$

where P_θ is a probability distribution indexed by a **parameter** θ , and Θ is the **parameter space** (the set of allowable parameter values).

Terminology checkpoint. A **parameter** θ belongs to the model/population; a **statistic** (e.g. a sample mean) is computed from the data to learn about θ . Remember: models are working assumptions to be checked.

Example 2.1 (Exponential model). Suppose we are interested in modelling **waiting times** until some event occurs (e.g. the time between customer arrivals at a service desk).

A simple model is to assume the data are i.i.d. from an exponential distribution:

$$X_i \mid \lambda \sim \text{Exp}(\lambda).$$

Write this formally as a **statistical model** and write down the **parameter space**.

Solution 2.1. Each data follows the exponential distribution with parameter λ . Since $\lambda > 0$, we have the model:

$$\mathcal{P} = \{\text{Exp}(\lambda) \mid \lambda > 0\},$$

with parameter $\theta = \lambda$ and parameter space¹

$$\Theta = \{\lambda \mid \lambda > 0\} = (0, \infty) = \mathbb{R}^+.$$

The exponential distribution is often used for modelling the time between events in a **Poisson process** — situations where events occur continuously and independently at a constant average rate.

¹**Set notation.** There are several equivalent ways to write sets:

- **Set-builder notation:** $\{\lambda \mid \lambda > 0\}$
- **Interval notation:** $(0, \infty)$
- **Named set:** $\mathbb{R}^+$

All three describe the same set. In this course, we recommend using **set-builder notation** when writing statistical models and their parameter spaces.

For example:

- the time between buses arriving at a stop if buses come at random,
- the time until the next radioactive decay of an atom,
- the waiting time until the next customer enters a shop.

In each case, the exponential model is a natural first assumption.

Example 2.2 (Normal model). Write down the **statistical model** and the corresponding **parameter space** used in Section 2.1.1.

Solution 2.2. We assumed the data are i.i.d. Normally distributed with *unknown mean* and *unknown variance*:

$$X_i \mid \mu, \sigma^2 \stackrel{\text{iid}}{\sim} \mathcal{N}(\mu, \sigma^2).$$

The **statistical model** is

$$\mathcal{P} = \{\mathcal{N}(\mu, \sigma^2) \mid \mu \in \mathbb{R}, \sigma^2 > 0\}.$$

Here there are two parameters, μ and σ^2 , which we group into a parameter vector $\underline{\theta} = (\mu, \sigma^2)$. The **parameter space** is²

$$\Theta = \{(\mu, \sigma^2) \mid \mu \in \mathbb{R}, \sigma^2 > 0\} = \mathbb{R} \times (0, \infty) = \mathbb{R} \times \mathbb{R}^+.$$

The normal model is widely used in practice, for example when data are approximately symmetric and unimodal, such as measurement errors in physics or biological traits like human heights.

²**Set notation (multiple parameters).** When a model has several parameters, it is best to use **set-builder notation** and collect the parameters into a vector

$$\underline{\theta} = (\theta_1, \theta_2, \dots, \theta_p).$$

The parameter space can then be written as

$$\Theta = \{(\theta_1, \dots, \theta_p) \mid \theta_1 \in S_1, \dots, \theta_p \in S_p\}.$$

Here each S_j is the set of allowed values for parameter θ_j . To construct Θ , simply **read off the constraints** on each parameter from the definition of the distribution (see the ?@sec-formula-sheet).

For example, in the Normal model:

- $\mu \in \mathbb{R}$ (any real number),
- $\sigma^2 \in (0, \infty)$ (variance must be positive),

so the parameter space is

$$\Theta = \mathbb{R} \times (0, \infty).$$

This is called a **Cartesian product**: we form the parameter space by taking all possible pairs (μ, σ^2) where the first component is from $\mathbb{R}$ and the second from $(0, \infty)$.

Intuitively, it's just “all combinations” of the allowed values for each parameter.

Example 2.3 (From words to a model (one-dimensional)). A questionnaire records whether each respondent would recommend a service: “yes” or “no”. Write down a simple model and identify the parameter.

Solution 2.3. Let $X_i = 1$ if respondent i says “yes”, and 0 otherwise.

Model $X_1, \dots, X_n \stackrel{\text{iid}}{\sim} \text{Bernoulli}(p)$, where $p = \Pr(X_i = 1)$ is the (unknown) long-run recommendation proportion.

Here $\theta = p$ and $\Theta = (0, 1)$. The **population** is all (current/future) potential respondents of interest; the **sample** is the n surveyed respondents.

Example 2.4 (A slightly richer example (multinomial)). A bag contains sweets of three colours {red, blue, green}. You draw n sweets at random *with replacement* and record counts (R, B, G) . Propose a model and name the parameter(s).

Solution 2.4. Model $(R, B, G) \sim \text{Multinomial}(n; p_R, p_B, p_G)$ with $p_R, p_B, p_G \geq 0$ and $p_R + p_B + p_G = 1$.

The parameter is $\theta = (p_R, p_B, p_G)$, and the parameter space is the 2-simplex $\Theta = \{(p_R, p_B, p_G) \in [0, 1]^3 : \sum p_i = 1\}$. The **population** is the long-run distribution of colours produced by the bag; the **sample** is the observed draws.

Example 2.5 (Image Classification from Machine Learning). In image classification (cat vs not-cat), each training example is a pair (X_i, Y_i) where X_i is an image and $Y_i \in \{0, 1\}$. A generic probabilistic model writes the conditional distribution

$$\Pr_{\theta}(Y = 1 \mid X = x) = g_{\theta}(x),$$

for some function family g_{θ} (e.g. logistic regression or a neural network). What are the sample, population, model, and parameter?

Solution 2.5.

- **Sample:** the labelled training set $\{(X_i, Y_i)\}_{i=1}^n$.
- **Population:** all (future) images and labels of interest.
- **Model:** the family of conditional distributions $\{\Pr_{\theta}(Y = 1 \mid X = x) = g_{\theta}(x)\}$.
- **Parameter:** θ (e.g. regression co

Note on Statistical Models

In this course we focus on **how to perform statistical inference once a statistical model has been specified**.

We will **assume the model is given**, and our task is to carry out inference by finding good parameter values using techniques such as:

- Parameter estimation
- Bayesian inference
- Interval estimation (e.g. confidence intervals, HDIs)
- Hypothesis testing

2.3 Exercises

1. For a study recording daily counts of emails received by a helpdesk, propose a one-parameter model and state the parameter space.
2. In a satisfaction survey with responses {poor, OK, good}, suggest a simple probabilistic model and identify the parameter(s).
3. State what “iid” means in one sentence, and give one example of how it could fail in practice.
4. Revisit {ref}stat-by-eye: if the spread of points increases, which parameter of $\mathcal{N}(\mu, \sigma^2)$ has changed, and how would you describe that change informally?

3 Types of Inference

Statistical inference is about using a **sample** to learn about a **population**. There are two fundamentally different philosophies of inference, and several types of inferential tasks that appear in both.

3.1 Philosophies of Inference

3.1.1 Frequentist Inference

Frequentist inference is based on the idea of **long-run frequency**.

- The **data** are considered random, as they represent one possible outcome of repeated sampling from the population.
- The **parameter** θ is fixed but unknown.
- Inference is about constructing procedures (estimators, tests, intervals) that perform well under repeated sampling.

Preview examples (we will return to these later): the sample mean $\hat{\mu}$ as an estimator of μ , a t -test for comparing means, or a 95% confidence interval.

3.1.2 Bayesian Inference

Bayesian inference is based on **subjective probability as degree of belief**.

- The **data** are fixed once observed.
- The **parameter** θ is treated as a random variable, with uncertainty described by a **prior distribution** $\pi(\theta)$.
- Inference updates this prior in light of the data, giving a **posterior distribution** $\pi(\theta \mid \text{data})$.

Preview examples (we will return to these later): a posterior mean as an estimator of μ , a 95% credible interval, or comparing hypotheses via posterior probabilities.

3.1.3 Key Differences

Perspective	Frequentist Inference	Bayesian Inference
Parameter	Fixed but unknown	Random variable with a prior
Sample	Random (from repeated sampling)	Fixed (once observed)
Probability	Long-run frequency	Degree of belief
Goal	Construct estimators, tests, CIs	Update beliefs via posterior
Prior	Ignored or minimal	Explicitly modelled via prior
Knowledge	(e.g. non-informative)	distribution
Output	Estimators, confidence intervals, p -values	Posterior distribution, credible intervals

3.2 Inference Tasks

Regardless of philosophy, the core tasks of inference are similar.

Suppose we want to infer a parameter θ from data:

1. **Point Estimation** Give a single “best guess” $\hat{\theta}$ for the parameter.

- Frequentist: choose an estimator with good properties (e.g. unbiasedness).
- Bayesian: often the posterior mean, median, or mode.

2. **Interval Estimation** Quantify uncertainty in θ by reporting a range.

- Frequentist: **confidence interval**

$$L \leq \theta \leq U$$

with a given long-run coverage probability.

- Bayesian: **credible interval**, the posterior probability that θ lies in $[L, U]$ is, say, 95%.

3. **Hypothesis Testing** {#sec-hypothesis-testing-intro} Ask a **yes/no question** about the parameter θ . This is done by formalising a claim (the *null hypothesis*) such as

$$H_0 : \theta \in \Theta_0,$$

and then using the data to judge whether this claim is **compatible with the evidence**.

- **Frequentist approach:** base the decision on a **test statistic** and its distribution under H_0 , leading to a **p -value** or a comparison with a **significance level**.

- **Bayesian approach:** compare hypotheses using **posterior probabilities** or **Bayes factors**, quantifying how much the data shift our beliefs.
4. **Prediction** Use data to predict future or unobserved outcomes.
- Frequentist: predictive distributions based on sampling models.
 - Bayesian: posterior predictive distribution, integrating over parameter uncertainty.

Part II

Part II — Parameter Estimation

4 Frequentist Point Estimation

A **statistic** is any function of the sample

$$T = T(X_1, \dots, X_n).$$

Because it is built from the random variables $X_1, \dots, X_n$, the statistic T itself is a **random quantity**: its value varies from sample to sample. Once the data have been observed $(x_1, \dots, x_n)$, plugging them into the same formula produces a single **non-random number**, which we denote by t_{obs} (or simply t).

When we pick a statistic for the specific purpose of approximating a parameter θ , we call it an **estimator**. Its realised value is the **estimate**.

Notation rule

- **Upper-case** letters $(T, \bar{X}, S^2)$ denote random statistics.
- **Lower-case** letters with a subscript “obs” $(t_{\text{obs}}, \bar{x}, s^2)$ denote the observed values of those statistics.

4.1 Estimators vs. Estimates

Concept	Notation	Description
Estimator	$\hat{\theta} = \hat{\theta}(X_1, \dots, X_n)$	A function of the data. A rule for producing an estimate.
Estimate	$\hat{\theta}_{\text{obs}} = \hat{\theta}(x_1, \dots, x_n)$	The numerical value you get after plugging in your data.

An estimator is a random variable; an estimate is its realised value.

4.1.1 Sample Mean and Variance

Definition 4.1. Sample Mean.

The sample mean of a **random sample** $X_1, \dots, X_n$ is

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i.$$

Definition 4.2. Sample Variance.

For a **random sample** $X_1, \dots, X_n$, the sample variance is

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2.$$

4.1.2 The Randomness of Estimators

The **randomness** of an estimator $\hat{\theta}$ is due to randomness of the sample being plugged into the function.

```
///  
echo: false  
  
mutable rngSeed = 1  
mutable meanHistory = []  
mutable resampleClicks = 0  
mutable resetClicks = 0  
mutable lastXbar = null // Store the previous xbar
```

```
///  
echo: false  
  
// Edge-trigger the Reset  
{  
  if (reset > resetClicks) {  
    mutable meanHistory = [];  
    mutable resetClicks = reset;  
    mutable lastXbar = null; // Also reset the stored xbar  
  }  
  return html``; // <- prevents "undefined" from being shown  
}  
  
// Edge-trigger the Resample: save the PREVIOUS mean before generating new sample  
{
```

```

if (resample > resampleClicks) {
  // If we have a previous mean stored, add it to history
  if (lastXbar !== null) {
    mutable meanHistory = [...meanHistory, lastXbar];
  }
  // Store current mean for next time
  mutable lastXbar = xbar;
  // Then bump seed for new sample
  mutable rngSeed = rngSeed + 1;
  mutable resampleClicks = resample;
}
return html``; // <- prevents "undefined" from being shown
}

```

```

//| echo: false

```

```

// --- Buttons at the top ---

```

```

// Helper to make a styled Inputs.button

```

```

function styledButton(label, cls, options = {}) {

```

```

  const el = Inputs.button(label, options);

```

```

  // <form>...<button>...</button></form>

```

```

  const btn = el.querySelector('button');

```

```

  if (btn) btn.classList.add(cls);

```

```

  // add your custom class

```

```

  return el;
}

```

```

// Define themed buttons

```

```

viewof resample = styledButton("Resample", "btn-resample", {

```

```

  reduce: c => (c ?? 0) + 1
});

```

```

viewof reset = styledButton("Reset", "btn-reset", {

```

```

  reduce: c => (c ?? 0) + 1
});

```

```

// viewof resample = Inputs.button("Resample", { reduce: c => (c ?? 0) + 1 })

```

```

// viewof reset = Inputs.button("Reset", { reduce: c => (c ?? 0) + 1 })

```

```

// --- Current sample depends on rngSeed + controls ---

```

```

X = {

```

```

  const rand = d3.randomNormal.source(d3.randomLcg(rngSeed))(controls.trueMu, controls.trueS

```

```

    return d3.range(controls.n).map(rand);
}
xbar = d3.mean(X)

// ===== Helpers =====
makeDensityReducer = (total) => (values, extent) =>
  values.length / Math.max(1, total) / Math.max(1e-9, extent.x2 - extent.x1);

// ===== Left panel: current sample (histogram + dots at y=0) =====
leftDensity = makeDensityReducer(controls.n)
xExtent = d3.extent(X)
xPad = 3 * controls.trueSigma
xDomain = [xExtent[0] - 0.1 * (xPad || 1), xExtent[1] + 0.1 * (xPad || 1)]

// === Compute yMax from actual binned densities ===
bins = d3.bin().domain(xDomain).thresholds(controls.bins)(X);
densities = bins.map(b =>
  (b.length / Math.max(1, controls.n)) / Math.max(1e-9, (b.x1 - b.x0))
);
yMax = d3.max(densities);
stickHeight = 0.1 * yMax; // 10% of the current y-range used by the histogram

leftPlot = Plot.plot({
  width: Math.min(600, Math.max(320, width/2 - 20)),
  height: 320,
  style: {
    background: "var(--plot-panel-bg)",
    color: "var(--brand-fg)" // drives 'currentColor'
  },
  x: { label: "Data", domain: xDomain },
  y: { label: "Density" },
  marks: [
    // histogram
    Plot.rectY(
      X,
      Plot.binX(
        { y: leftDensity },
        {
          x: d => d,
          thresholds: controls.bins,
          inset: 0,
          fillOpacity: 0.4,

```



```

        fill: "var(--brand-teal)"
    }
)
),
// raw points
// Plot.dot(
//   X.map(x => ({ x, y: 0 })),
//   {
//     x: "x",
//     y: "y",
//     r: 2,
//     fillOpacity: 0.9,
//     fill: "var(--brand-teal)",
//     symbol: "cross"
//   }
// ),
Plot.ruleX(
  X,
  { y1: 0, y2: yMax * 0.05, stroke: "var(--brand-teal)", strokeWidth: 3, strokeOpacity: 1.
},
// xbar reference
Plot.ruleX([xbar], { stroke: "currentColor", strokeDasharray: "4,3" }),
Plot.dot([x: xbar, y: 0 ], { x: "x", y: "y", r: 3, fill: "currentColor" }),
Plot.ruleY([0])
]
})

// ===== Right panel: saved sample means (strip + hist + optional true curve) =====
means = meanHistory
rightDensity = makeDensityReducer(means.length)

mPad = 3 * controls.trueSigma / Math.sqrt(controls.n)
meansDomain = means.length ? d3.extent(means) : [controls.trueMu - mPad, controls.trueMu + m
mDomain = [
  Math.min(meansDomain[0], controls.trueMu - mPad),
  Math.max(meansDomain[1], controls.trueMu + mPad)
]

sampleMeanPdf = (x) =>
  (1 / Math.sqrt(2 * Math.PI * (controls.trueSigma**2 / controls.n))) *
  Math.exp(-((x - controls.trueMu)**2) / (2 * (controls.trueSigma**2 / controls.n)))

```

```

pdfXs = d3.scaleLinear().domain(mDomain).ticks(200)
pdfData = pdfXs.map(x => ({ x, y: sampleMeanPdf(x) }))

rightPlot = Plot.plot({
  width: Math.min(600, Math.max(320, width/2 - 20)),
  height: 320,
  style: {
    // make the figure match your theme
    background: "var(--plot-panel-bg)",
    color: "var(--brand-fg)" // this drives 'currentColor'
  },
  x: { label: " $\bar{x}$ ", domain: mDomain },
  y: { label: "Density" },
  marks: [
    Plot.rectY(
      means,
      Plot.binX(
        { y: rightDensity },
        { x: d => d, thresholds: controls.bins, inset: 0, fillOpacity: 0.3,
          fill: "var(--brand-red)" }
      )
    ),
    Plot.dot(means.map(m => ({ x: m, y: 0 })), {
      x: "x", y: "y", r: 2, fillOpacity: 0.7, fill: "var(--brand-red)"
    }),
    Plot.dot([x: xbar, y: 0], {
      x: "x", y: "y", r: 4, fill: "currentColor" // uses style.color above
    }),
    ...(controls.showTrue
      ? [Plot.line(pdfData, { x: "x", y: "y", strokeWidth: 2, stroke: "var(--brand-red)" })]
      : []
    ),
    Plot.ruleY([0])
  ]
})

// ===== Display =====

// html`<div style="
//   display:flex;
//   align-items:center;
//   gap:10px;

```

```

//   margin-bottom:6px;
//   width:100%;
// ">
//   <span style="display:inline-flex;">${viewof resample}</span>
//   <span style="flex:1 1 auto; min-width:100px;"></span>  <!-- flexible spacer that can sh
//   <span style="display:inline-flex;">${viewof reset}</span>
// </div>`
{
  // Make the forms behave nicely in the grid
  for (const f of [viewof resample, viewof reset]) {
    f.style.margin = "0";
    f.style.width = "100%";      // inline style beats the library rule
    f.style.maxWidth = "none";
    f.style.display = "flex";    // so we can push the button left/right
  }
  // Left button to the left, right button to the right
  viewof resample.style.justifyContent = "flex-start";
  viewof reset.style.justifyContent   = "flex-end";

  return html`<div class="ojs-toolbar-grid">
    <div class="left">${viewof resample}</div>
    <div class="center">
      ${md`${tex`\bar{x} = ${xbar.toFixed(2)}\`}`}
    </div>
    <div class="right">${viewof reset}</div>
  </div>

  <style>
    .ojs-toolbar-grid{
      display:grid;
      grid-template-columns:1fr auto 1fr; /* left, middle, right */
      align-items:center;
      width:100%;
      box-sizing:border-box;
      margin:0; padding:0;
    }
    .ojs-toolbar-grid form{
      margin:0 !important;
      width:100% !important;
      max-width:none !important;
      display:flex !important;
    }
  </style>

```

```

.ojs-toolbar-grid .left form{ justify-content:flex-start !important; }
.ojs-toolbar-grid .right form{ justify-content:flex-end !important; }

.btn.btn-resample{ background: var(--brand-teal); color:#fff; border:none; }
.btn.btn-resample:hover{ background: var(--brand-teal-hover); }
.btn.btn-reset{ background: var(--brand-red); color:#fff; border:none; }
.btn.btn-reset:hover{ background: var(--brand-red-hover); }
.btn.btn-resample:focus-visible,
.btn.btn-reset:focus-visible{
  outline: 2px solid currentColor;
  outline-offset: 2px;
}

.ojs-toolbar-grid .center{
  text-align:center;
  font-size:0.9rem;
  color: var(--brand-fg);
}
</style>`;
}

```

```

html`<div style="max-width:100%; display:flex; gap:14px; align-items:flex-start; flex-wrap:n
  <div>${leftPlot}</div>
  <div>${rightPlot}</div>
</div>`

```

```

html`<style>
details.controls-root > summary {
  list-style: none; cursor: pointer; user-select: none;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: 10px;
  padding: 8px 10px;
  font-weight: 600; font-size: 13px;
  color: var(--fg-strong);
  display: flex; align-items: center; gap: 8px;
}
details.controls-root > summary::-webkit-details-marker { display: none; }
details.controls-root[open] > summary { border-bottom-left-radius: 0; border-bottom-right-ra

```

```

/* ----- Compact Inputs (fixed slider alignment) ----- */
.controls-col label { font-size: 12px !important; }

.controls-col input[type="range"] {
  -webkit-appearance: none;
  appearance: none;
  width: 100%;
  height: 18px; /* overall control box height */
  background: transparent;
  vertical-align: middle;
  accent-color: var(--brand-teal, #55C3CB);
}

/* WebKit track */
.controls-col input[type="range"]::-webkit-slider-runnable-track {
  height: 4px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 35%, transparent);
}

/* WebKit thumb */
.controls-col input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 12px; height: 12px; border-radius: 50%;
  background: var(--brand-teal, #55C3CB);
  border: 0;
  /* Center the 12px thumb over the 4px track: (4 - 12)/2 = -4px */
  margin-top: -4px;
}

/* Firefox track */
.controls-col input[type="range"]::-moz-range-track {
  height: 4px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 35%, transparent);
}

/* Firefox thumb (auto-centred; no negative margin needed) */
.controls-col input[type="range"]::-moz-range-thumb {
  width: 12px; height: 12px; border-radius: 50%;
  background: var(--brand-teal, #55C3CB);
}

```

```

border: 0;
}

/* Optional: hide the default filled "progress" colour in Firefox */
.controls-col input[type="range"]::-moz-range-progress {
  height: 4px; border-radius: 999px;
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 35%, transparent);
}

.controls-grid {
  display: grid;
  grid-template-columns: minmax(260px,1fr) minmax(260px,1fr);
  gap: 12px;
  border: 1px solid var(--border); border-top: none;
  border-bottom-left-radius: 10px; border-bottom-right-radius: 10px;
  padding: 10px;
  background: var(--surface-weak);
  overflow-x: auto;
}

.controls-col { min-width: 260px; }
.controls-col h4 {
  margin: 0 0 6px 0; font-size: 12px; font-weight: 700;
  color: var(--fg-muted);
}

/* chevron */
details.controls-root > summary::before {
  content: ""; display: inline-block; margin-right: 8px; width: 0; height: 0;
  border-style: solid; border-width: 3px 0 3px 6px;
  border-color: transparent transparent transparent var(--fg-strong);
  transform: rotate(0deg); transform-origin: 3px 50%; transition: transform 50ms ease;
}
details.controls-root[open] > summary::before { transform: rotate(90deg); }
</style>`

/* ----- One merged, collapsible control ----- */
viewof controls = {

  // Left column: data & truth
  const leftForm = Inputs.form({
    n: Inputs.range([1, 400], { value: 10, step: 1, label: md`${tex`n`}` },),
    trueMu: Inputs.range([-2, 2], { value: 0, step: 0.1, label: md`${tex`\mu_{\text{true}}}` }

```

```

    trueSigma: Inputs.range([0.3, 3], { value: 1, step: 0.1, label: md`${tex`\sigma_{\text{t}}`}, { submit: false });

// Right column: display & actions
const rightForm = Inputs.form({
  bins: Inputs.range([10, 60], { value: 24, step: 1, label: "Histogram bins" }),
  showTrue: Inputs.toggle({ label: md`Show ${tex`\bar{X}`} distribution`, value: false }),
  // include the buttons as inputs so their *click counts* become part of form.value
}, { submit: false });

// Collapsible wrapper
const root = html`<details class="controls-root" open>
  <summary>Simulation controls</summary>
  <div class="controls-grid">
    <div class="controls-col"><h4>Data & truth</h4></div>
    <div class="controls-col"><h4>Display & actions</h4></div>
  </div>
</details>`;
root.querySelector(".controls-grid").children[0].append(leftForm);
root.querySelector(".controls-grid").children[1].append(rightForm);

// Merge both forms into a single value object
const getValue = () => ({ ...leftForm.value, ...rightForm.value });
const update = () => {
  root.value = getValue();
  root.dispatchEvent(new CustomEvent("input", { bubbles: true }));
};

leftForm.addEventListener("input", update);
rightForm.addEventListener("input", update);
// make sure button clicks trigger updates even if no 'input' is fired

queueMicrotask(update);
return root;
}

```

4.2 Bias, Variance, MSE and Consistency

Definition 4.3. Bias.

The bias of an estimator $\hat{\theta}$ for parameter θ is

$$\text{Bias}(\hat{\theta}) = \mathbb{E}[\hat{\theta}] - \theta.$$

If $\text{Bias}(\hat{\theta}) = 0$, the estimator is **unbiased**.

Definition 4.4. Variance.

The variance of an estimator $\hat{\theta}$ is simply

$$\text{Var}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \mathbb{E}[\hat{\theta}])^2].$$

Definition 4.5. Mean Squared Error (MSE).

The MSE of $\hat{\theta}$ is defined as

$$\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \theta)^2] = \text{Var}(\hat{\theta}) + \text{Bias}(\hat{\theta})^2.$$

Definition 4.6. Consistency.

An estimator $\hat{\theta}_n$ (depending on sample size n) is **consistent** if

$$\hat{\theta}_n \xrightarrow{p} \theta \quad \text{as } n \rightarrow \infty,$$

i.e. it converges in probability to the true parameter value.

4.2.1 Examples

Example 4.1 (Bias of the Sample Mean). For $X_1, \dots, X_n \stackrel{iid}{\sim}$ distribution with mean μ :

$$\mathbb{E}[\bar{X}] = \mu \quad \implies \quad \text{Bias}(\bar{X}) = 0.$$

So the sample mean is **unbiased** for μ .

Example 4.2 (Variance of the Sample Mean). If $\text{Var}(X_i) = \sigma^2$:

$$\text{Var}(\bar{X}) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{\sigma^2}{n}.$$

So as n grows, the estimator becomes more concentrated around μ .

Example 4.3 (Bias of the Sample Variance). If we instead used the “naïve” variance

$$s_{naive}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2,$$

then

$$\mathbb{E}[s_{naive}^2] = \frac{n-1}{n} \sigma^2,$$

which underestimates σ^2 .

That is why we divide by $n-1$ instead of n .

Example 4.4 (MSE of Sample Mean). Suppose we estimate μ with $\hat{\mu} = c\bar{X}$, where c is a constant.

- Bias:

$$\text{Bias}(\hat{\mu}) = (c-1)\mu.$$

- Variance:

$$\text{Var}(\hat{\mu}) = c^2 \frac{\sigma^2}{n}.$$

- MSE:

$$\text{MSE}(\hat{\mu}) = \frac{c^2 \sigma^2}{n} + (c-1)^2 \mu^2.$$

This shows how a biased estimator can still have low MSE if variance is reduced.

Example 4.5 (Consistency of $\bar{X}$). By the **Law of Large Numbers**,

$$\bar{X} \xrightarrow{p} \mu,$$

so the sample mean is a **consistent** estimator of μ .

4.3 Central Limit Theorem (CLT)

Definition 4.7 (Central Limit Theorem). If $X_1, \dots, X_n \stackrel{iid}{\sim}$ distribution with mean μ and variance $\sigma^2 < \infty$, then

$$\frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \xrightarrow{d} N(0, 1).$$

The CLT explains why many estimators (like $\bar{X}$) are approximately normal in large samples, which is crucial for constructing confidence intervals and hypothesis tests.

5 Likelihood

The likelihood function is the **joint density (or probability mass) function** of the data:

$$f(\underline{x} \mid \theta), \quad \underline{x} = (x_1, \dots, x_n).$$

5.1 From Joint Density to Likelihood

Suppose we observe n independent and identically distributed (i.i.d.) samples,

$$X_1, \dots, X_n \stackrel{\text{iid}}{\sim} f_X(x \mid \theta).$$

Then the **joint density** is given by multiplying the individual densities:

$$f(\underline{x} \mid \theta) = \prod_{i=1}^n f_X(x_i \mid \theta).$$

Remark 5.1 (Why products?).

- Independence means that joint probabilities factorise into products:

$$P(A \cap B) = P(A) P(B).$$

- By the same rule, the joint density of independent random variables is the product of their individual densities.
- This is often the first place where students forget the independence assumption! Without it, you **cannot** just multiply.

5.2 Likelihood Function

Definition 5.1. Likelihood Function. For fixed observed data $\underline{x}$, the **likelihood function** is defined as

$$L(\theta; \underline{x}) := f(\underline{x} \mid \theta).$$

Here we treat θ as the **variable**, and the data $\underline{x}$ as fixed.

💡 Likelihood vs. Probability:

- **PDF/PMF:** function of $\underline{x}$ (data random, parameter fixed).
- **Likelihood:** function of θ (data fixed, parameter variable).

Visualisation:

```
///| echo: false
// --- Data & stats ---

html`<style>
.plot--large-axis .x-axis .label,
.plot--large-axis .y-axis .label {
font-size: 24px; font-weight: 600;
}
.plot--large-axis .x-axis text,
.plot--large-axis .y-axis text {
font-size: 12px; /* tick labels (optional) */
}
</style>`

// Only resimulate when resim button is clicked or data parameters change
X = {
  // Trigger on: resim button, n, trueMu, or trueSigma changes
  resim; n; trueMu; trueSigma;

  const m = trueMu, s = trueSigma;
  const out = new Array(n);
  for (let i = 0; i < n; i += 2) {
```

```

    const u = Math.random(), v = Math.random();
    const R = Math.sqrt(-2 * Math.log(u)), T = 2 * Math.PI * v;
    const z0 = R * Math.cos(T), z1 = R * Math.sin(T);
    out[i] = m + s * z0;
    if (i + 1 < n) out[i + 1] = m + s * z1;
  }
  return out;
};

xbar = d3.mean(X);
S = d3.sum(X, x => (x - xbar) ** 2);

logLik = (mu, sigma2) => {
  const ss = d3.sum(X, x => (x - mu) ** 2);
  return -0.5 * n * Math.log(2 * Math.PI * sigma2) - 0.5 * ss / sigma2;
};

muhat = xbar;
sigma2hat = S / n;

// --- Grid for contours (axes ticks) ---
// These only need to update when display parameters change
muGrid = d3.ticks(muhat - muHalfWidth, muhat + muHalfWidth, gridN)
sig2Grid = d3.ticks(Math.max(0.05, sigma2hat / 20), sig2Max, gridN).filter(s => s > 0)

// --- Build a row-major grid for Plot.raster / Plot.contour ---
nx = muGrid.length
ny = sig2Grid.length

// Memoize the expensive log-likelihood computation
llValues = {
  const z = new Float64Array(nx * ny);
  for (let j = 0; j < ny; ++j) {
    for (let i = 0; i < nx; ++i) {
      const s2 = Math.max(1e-6, sig2Grid[j]);
      z[j * nx + i] = logLik(muGrid[i], s2);
    }
  }
  return z;
}

```

```

// ---- Geometry / domains (responsive) ----
gap = 20
containerW = width
plotW = Math.round(Math.max(240, Math.min(600, (containerW - gap) / 2 - 15)))
plotH = Math.round(plotW * 0.8)

marginTop = 20
marginRight = 35
marginBottom = 20
marginLeft = 35
innerW = plotW - marginLeft - marginRight
innerH = plotH - marginTop - marginBottom

muDomain = [muGrid[0], muGrid[muGrid.length - 1]]
sig2Domain = [sig2Grid[0], sig2Grid[sig2Grid.length - 1]]

// ---- Colour scaling (robust & reactive) ----
llMax = d3.max(llValues)
llRel = Float64Array.from(llValues, v => v - llMax)

// Use more efficient quantile computation
llRelFinite = Array.from(llRel).filter(Number.isFinite)
llRelSorted = llRelFinite.slice().sort(d3.ascending)
q05 = d3.quantileSorted(llRelSorted, 0.05) ?? d3.min(llRelFinite)

colorDomain = [q05, 0]
llRelClipped = Float64Array.from(llRel, v => Math.max(q05, v))
contourThresholds = d3.ticks(q05, 0, 10)

bodyFont = getComputedStyle(document.body).fontFamily

// ---- Left panel: raster (clipped) + contour (same grid) ----
viewof leftPanel = {
  const wrap = html`<div style="
    position: relative;
    width: ${plotW}px;
    height: ${plotH}px;
    flex: 0 0 ${plotW}px;
    line-height: 0;
  "></div>`;

  const base = Plot.plot({

```

```

width: plotW, height: plotH,
marginLeft, marginRight, marginTop, marginBottom,
style: {
  background: "var(--brand-bg)",
  color: "var(--brand-fg)",
  fontFamily: bodyFont,
  fontSize: 14
},
x: { label: " ", domain: muDomain },
y: { label: "²", domain: sig2Domain },
color: {
  type: "symlog",
  domain: colorDomain,
  clamp: true,
  scheme: "turbo",
  legend: false
},
marks: [
  Plot.raster(11RelClipped, {
    width: nx, height: ny,
    x1: muDomain[0], x2: muDomain[1],
    y1: sig2Domain[0], y2: sig2Domain[1],
    interpolate: "nearest"
  }),
  Plot.dot([ { mu: muhat, sig2: sigma2hat } ], {
    x: "mu", y: "sig2", r: 5, stroke: "currentColor", fill: "var(--brand-bg)"
  })
]
});

base.classList.add("plot--large-axis");
base.style.display = "block";
wrap.append(base);

// Crosshair overlay fills wrapper
const svg = d3.create("svg")
  .attr("width", plotW).attr("height", plotH)
  .style("position", "absolute").style("left", "0").style("top", "0")
  .style("width", "100%").style("height", "100%")
  .style("pointer-events", "none");

```

```

const xScale = d3.scaleLinear().domain(muDomain).range([marginLeft, marginLeft + innerW]);
const yScale = d3.scaleLinear().domain(sig2Domain).range([marginTop + innerH, marginTop]);

const vline = svg.append("line")
  .attr("x1", xScale(muhat)).attr("x2", xScale(muhat))
  .attr("y1", marginTop).attr("y2", marginTop + innerH)
  .attr("stroke", "currentColor");

const hline = svg.append("line")
  .attr("x1", marginLeft).attr("x2", marginLeft + innerW)
  .attr("y1", yScale(sigma2hat)).attr("y2", yScale(sigma2hat))
  .attr("stroke", "currentColor");

wrap.append(svg.node());

// Hitbox fills wrapper; subtract margins in code
const hit = html`<div style="
  position:absolute; left:0; top:0; width:100%; height:100%;
  cursor:crosshair; background:transparent; touch-action:none;
"></div>`;
wrap.append(hit);

const state = { x: muhat, y: sigma2hat, dragging: false };
wrap.value = { ...state }; // <-- expose full state via the view's value

function emit() {
  wrap.value = { ...state }; // <-- assign a fresh object so equality changes
  wrap.dispatchEvent(new CustomEvent("input", { bubbles: true }));
}

function setFromEvent(evt) {
  const r = hit.getBoundingClientRect();
  let px = evt.clientX - r.left, py = evt.clientY - r.top;
  px = Math.min(Math.max(0, px - marginLeft), innerW);
  py = Math.min(Math.max(0, py - marginTop), innerH);
  state.x = muDomain[0] + (px / innerW) * (muDomain[1] - muDomain[0]);
  state.y = sig2Domain[1] - (py / innerH) * (sig2Domain[1] - sig2Domain[0]);
  vline.attr("x1", xScale(state.x)).attr("x2", xScale(state.x));
  hline.attr("y1", yScale(state.y)).attr("y2", yScale(state.y));
  emit(); // <-- notify dependents
}

```



```

// RAF throttle for move
let raf = 0;
function scheduleUpdate(e) {
  if (raf) return;
  raf = requestAnimationFrame(() => { raf = 0; setFromEvent(e); });
}

function setDragging(on) {
  if (state.dragging === on) return;
  state.dragging = on;
  emit(); // <-- notify on drag state changes too
}

hit.addEventListener("pointerdown", (e) => {
  setDragging(true);
  setFromEvent(e);
  hit.setPointerCapture(e.pointerId);
});

hit.addEventListener("pointermove", (e) => {
  if (state.dragging) scheduleUpdate(e);
});

const endDrag = () => setDragging(false);
hit.addEventListener("pointerup", endDrag);
hit.addEventListener("pointercancel", endDrag);
hit.addEventListener("pointerleave", endDrag);

// initial notify
queueMicrotask(emit);

return wrap; // <-- the view element
}

// isDragging = leftPanel.dragging ?? false
// console.log(isDragging)
// numPts = isDragging ? 100 : 200 // fewer x-samples while dragging
// pdfStroke = isDragging ? 1 : 2 // thinner line while dragging

// // Read probe values reactively (no extra visible block)

```

```

// probeRaw = Generators.input(leftPanel.hit);
// probeMu = (Array.isArray(probeRaw) ? probeRaw[0] : probeRaw?.x) ?? muhat;
// probeSig2 = (Array.isArray(probeRaw) ? probeRaw[1] : probeRaw?.y) ?? sigma2hat;
// probeLL = logLik(probeMu, probeSig2);

// leftPanel is now { x, y, dragging }
isDragging = leftPanel.dragging
numPts      = isDragging ? 100 : 200
pdfStroke   = isDragging ? 1   : 2

// if you want the probe to follow the crosshair:
probeMu     = leftPanel.x
probeSig2   = leftPanel.y
probeLL     = logLik(probeMu, probeSig2);

// --- Right panel ---
normalPdf = (x, mu, sig2) => (1 / Math.sqrt(2 * Math.PI * sig2)) * Math.exp(-((x - mu) ** 2))

xExtent = d3.extent(X)
xPad = 3 * Math.sqrt(probeSig2)
xMin = Math.min(xExtent[0], probeMu - xPad)
xMax = Math.max(xExtent[1], probeMu + xPad)

// PDF curve
xs = d3.range(numPts).map(i => xMin + (i / (numPts - 1)) * (xMax - xMin))
pdfYs = xs.map(x => normalPdf(x, probeMu, probeSig2))

// Density reducer
densityReducer = (values, extent) => values.length / n / (extent.x2 - extent.x1)

// Compute yMax
bins_ = d3.bin().domain([xMin, xMax]).thresholds(bins)(X);
densities = bins_.map(b =>
  (b.length / Math.max(1, n)) / Math.max(1e-9, (b.x1 - b.x0))
);
yMax = d3.max(densities);

rightPanel = Plot.plot({
  width: plotW, height: plotH,
  marginLeft, marginBottom,

```

```

x: { label: "Data", domain: [xMin, xMax] },
y: { label: "Density" },
style: {
  fontFamily: bodyFont,
  fontSize: 14.
},
marks: [
  Plot.rectY(
    X,
    Plot.binX(
      { y: densityReducer },
      {
        x: d => d,
        thresholds: bins,
        inset: 0,
        fill: "var(--brand-red)",
        fillOpacity: 0.2
      }
    )
  ),
  Plot.line(xs.map((x, i) => ({ x, y: pdfYs[i] })), {
    x: "x", y: "y",
    stroke: "var(--brand-red)",
    strokeWidth: pdfStroke
  }),
  Plot.ruleX(X, {
    y1: 0,
    y2: yMax * 0.05,
    stroke: "var(--brand-red)",
    strokeWidth: 2,
    strokeOpacity: 0.8
  }),
  Plot.ruleY([0]),
  Plot.ruleX([probeMu], { stroke: "currentColor", strokeDasharray: "4,3" })
]
})

probeInfo = md`**Probe:**  $\mu=${probeMu.toFixed(2)}, \sigma^2=${probeSig2.toFixed(3)}$ 

html`<div style="max-width:100%;">
<div style="display:flex; gap:${gap}px; align-items:flex-start; flex-wrap:nowrap; overflow-x:
<div style="flex:0 0 ${plotW}px;">${viewof leftPanel}</div>

```

```

<div style="flex:0 0 ${plotW}px;">${rightPanel}</div>
</div>
<div style="margin-top:10px; margin-bottom:-10px">${probeInfo}</div>
</div>`

```

```

//| echo: false

```

```

// --- Styles: compact, 2-column, collapsible, custom button ---
html`<style>

```

```

/* ----- Collapsible container ----- */

```

```

details.controls-root > summary {

```

```

    list-style: none; cursor: pointer; user-select: none;

```

```

    background: var(--surface, #f1f5f9);

```

```

    border: 1px solid var(--border, #e5e7eb);

```

```

    border-radius: 10px;

```

```

    padding: 8px 10px;

```

```

    font-weight: 600; font-size: 13px;

```

```

    color: var(--fg-strong, #334155);

```

```

    display: flex; align-items: center; gap: 8px;

```

```

}

```

```

details.controls-root > summary::-webkit-details-marker { display: none; }

```

```

details.controls-root[open] > summary { border-bottom-left-radius: 0; border-bottom-right-ra

```

```

/* ----- Two-column grid ----- */

```

```

.controls-grid {

```

```

    display: grid;

```

```

    grid-template-columns: minmax(260px, 1fr) minmax(260px, 1fr);

```

```

    gap: 12px;

```

```

    border: 1px solid var(--border, #e5e7eb); border-top: none;

```

```

    border-bottom-left-radius: 10px; border-bottom-right-radius: 10px;

```

```

    padding: 10px;

```

```

    background: var(--surface-weak, #fafafa);

```

```

    overflow-x: auto; /* prefer scroll over wrapping on narrow screens */

```

```

}

```

```

.controls-col { min-width: 260px; }

```

```

.controls-col h4 {

```

```

    margin: 0 0 6px 0;

```

```

    font-size: 12px; font-weight: 700;

```

```

    color: var(--fg-muted, #475569);

```

```

}

```

```

/* ----- Compact Inputs (fixed slider alignment) ----- */
.controls-col label { font-size: 12px !important; }

.controls-col input[type="range"] {
  -webkit-appearance: none;
  appearance: none;
  width: 100%;
  height: 18px; /* overall control box height */
  background: transparent;
  vertical-align: middle;
  accent-color: var(--brand-teal, #55C3CB);
}

/* WebKit track */
.controls-col input[type="range"]::-webkit-slider-runnable-track {
  height: 4px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 35%, transparent);
}

/* WebKit thumb */
.controls-col input[type="range"]::-webkit-slider-thumb {
  -webkit-appearance: none;
  appearance: none;
  width: 12px; height: 12px; border-radius: 50%;
  background: var(--brand-teal, #55C3CB);
  border: 0;
  /* Center the 12px thumb over the 4px track: (4 - 12)/2 = -4px */
  margin-top: -4px;
}

/* Firefox track */
.controls-col input[type="range"]::-moz-range-track {
  height: 4px;
  border-radius: 999px;
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 35%, transparent);
}

/* Firefox thumb (auto-centred; no negative margin needed) */
.controls-col input[type="range"]::-moz-range-thumb {
  width: 12px; height: 12px; border-radius: 50%;
  background: var(--brand-teal, #55C3CB);
}

```

```

border: 0;
}

/* Optional: hide the default filled "progress" colour in Firefox */
.controls-col input[type="range"]::-moz-range-progress {
  height: 4px; border-radius: 999px;
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 35%, transparent);
}

/* ----- Button ----- */
.btn-resim button {
  background: var(--brand-teal, #55C3CB);
  color: #fff;
  border: none;
  border-radius: 6px;
  padding: 6px 12px;
  font-weight: 600;
  font-size: 12px;
  cursor: pointer;
  transition: filter 120ms ease, background-color 120ms ease;
}

/* hover effect */
.btn-resim button:hover {
  background: color-mix(in srgb, var(--brand-teal, #55C3CB) 88%, black);
}

/* chevron */
details.controls-root > summary::before {
  content: ""; display: inline-block; margin-right: 8px; width: 0; height: 0;
  border-style: solid; border-width: 3px 0 3px 6px;
  border-color: transparent transparent transparent var(--fg-strong);
  transform: rotate(0deg); transform-origin: 3px 50%; transition: transform 50ms ease;
}
details.controls-root[open] > summary::before { transform: rotate(90deg); }
</style>`

// --- Build one merged, collapsible control "input" ---
viewof controls = {
  const resimBtn = Inputs.button("Resimulate data");
  resimBtn.classList.add("btn-resim");

```

```

const leftForm = Inputs.form({
  n: Inputs.range([1, 400], { value: 10, step: 1, label: md`${tex`n`}` }),
  trueMu: Inputs.range([-3, 3], { value: 0.5, step: 0.05, label: md`${tex`\mu_{\text{true}}`}` }),
  trueSigma: Inputs.range([0.3, 3], { value: 1.2, step: 0.05, label: md`${tex`\sigma_{\text{true}}`}` }),
}, { submit: false });

const rightForm = Inputs.form({
  muHalfWidth: Inputs.range([1, 5], { value: 2.5, step: 0.05, label: md`${tex`\mu\text{-range}`}` }),
  sig2Max: Inputs.range([0.2, 6], { value: 3, step: 0.05, label: md`${tex`\sigma^2\text{max}`}` }),
  gridN: Inputs.range([25, 121], { value: 81, step: 4, label: "Grid size" }),
  bins: Inputs.range([10, 60], { value: 24, step: 1, label: "Histogram bins" })
}, { submit: false });

const root = html`<details class="controls-root" open>
  <summary>Simulation controls</summary>
  <div class="controls-grid">
    <div class="controls-col"><h4>Data & truth</h4></div>
    <div class="controls-col"><h4>Grid & display</h4></div>
  </div>
</details>`;

const grid = root.querySelector(".controls-grid");
const leftCol = grid.children[0];
const rightCol = grid.children[1];

leftCol.append(leftForm);

rightCol.append(rightForm);
const btnWrap = html`<div style="display:flex; justify-content:flex-start; margin-top:6px;">
  <button type="button">Resim</button>
</div>`;
btnWrap.append(resimBtn);
rightCol.append(btnWrap);

// --- Merge values
const getValue = () => ({ ...leftForm.value, ...rightForm.value, resim: resimBtn.value });
const update = () => {
  root.value = getValue();
  root.dispatchEvent(new CustomEvent("input", { bubbles: true }));
};

// ONE listener for all children (sliders + button)
grid.addEventListener("input", update);

```

```

    queueMicrotask(update);
    return root;
}

// --- Expose reactive vars
n = controls.n
trueMu = controls.trueMu
trueSigma = controls.trueSigma
muHalfWidth = controls.muHalfWidth
sig2Max = controls.sig2Max
gridN = controls.gridN
bins = controls.bins
resim = controls.resim

```

5.2.1 Computing Likelihood Functions

This is a **very important** part of the course.

Example 5.1 (Bernoulli Likelihood). Suppose we have n i.i.d. data distributed according to the `?@sec-bern`:

$$Y_i \mid p \sim \text{Bernoulli}(p).$$

Derive the **likelihood function**.

Example 5.2 (Bernoulli Likelihood). Suppose $n = 5$ trials give data $\underline{x} = (1, 0, 1, 1, 0)$. Then $\sum_{i=1}^n x_i = 3$.

$$L(p; \underline{x}) = p^3(1 - p)^2.$$

- If $p = 0.5$, $L(0.5) = 0.5^3 \cdot 0.5^2 = 0.03125$.
- If $p = 0.7$, $L(0.7) = 0.7^3 \cdot 0.3^2 \approx 0.0309$.
- If $p = 0.8$, $L(0.8) = 0.8^3 \cdot 0.2^2 = 0.0205$.

So the data are slightly more likely under $p = 0.5$ than $p = 0.8$.

5.3 Maximum Likelihood Estimation

Definition 5.2 (Maximum Likelihood Estimator (MLE)). The MLE is the parameter value $\hat{\theta}$ that maximises the likelihood function:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta \in \Theta} L(\theta \mid \underline{x}).$$

Because logs are monotone, it is usually easier to maximise the **log-likelihood**:

$$\ell(\theta; \underline{x}) = \log L(\theta; \underline{x}).$$

5.3.1 Computing MLEs

Example 5.3 (Bernoulli MLE). From Example 5.1, we have

$$L(\theta; x)$$

Compute the MLE of p .

Solution 5.1. From the provided likelihood, we first compute the **log-likelihood**:

$$\ell(p \mid \underline{x}) = \left(\sum_i x_i \right) \log p + \left(n - \sum_i x_i \right) \log(1 - p).$$

Differentiate and solve:

$$\frac{\partial}{\partial p} \ell(p \mid \underline{x}) = \frac{\sum_i x_i}{p} - \frac{n - \sum_i x_i}{1 - p} = 0.$$

This gives

$$\hat{p}_{\text{MLE}} = \frac{1}{n} \sum_{i=1}^n x_i,$$

the **sample proportion of successes**.

Example 5.4 (Binomial MLE). Suppose we have a **single ?@sec-binom** observation

$$X \mid \theta \sim \text{Bin}(n, \theta).$$

Derive its **likelihood function**.

Solution 5.2. Since we only have **one** observation, the likelihood function is just the distribution of the observation:

$$L(\theta; x) = f(x | \theta) = \binom{n}{x} \theta^x (1 - \theta)^{n-x}.$$

The log-likelihood is thus

$$\ell(\theta; x) = \log f(x | \theta) = \log \left(\binom{n}{x} \right) + x \log \theta + (n - x) \log(1 - \theta).$$

Differentiating¹, we obtain

$$\frac{\partial}{\partial \theta}$$

Example 5.5 (Binomial MLE). Suppose we have n **independent** Binomial data:

$$X_i | \theta \sim \text{Bin}(n_i, \theta).$$

Note here that each Binomial data has a different number of Bernoulli trials n_i , which are known.

Derive its **likelihood function**.

Solution 5.3. The likelihood function is the joint distribution of the n observations

$$f(\underline{x} | \theta) = \prod_{i=1}^n \binom{n_i}{x_i} \theta^{x_i} (1 - \theta)^{n_i - x_i}.$$

✦ Big picture:

- Likelihood tells us how **plausible** each parameter value is, given the data.
- The MLE chooses the parameter value that makes the observed data most likely.
- Later, in Bayesian inference, we will combine the likelihood with a **prior distribution** on θ to obtain a **posterior distribution**.

5.4 Summary

We can compute likelihood functions

¹Differentiating the log.

6 Bayesian Inference

```
import matplotlib.pyplot as plt

plot_bg = "#f8f8f8"

plt.rcParams["figure.facecolor"] = plot_bg
plt.rcParams["axes.facecolor"] = plot_bg
```

6.1 Introduction to the Bayesian approach

As discussed in the course introduction, **Bayesian inference** takes a different perspective from the frequentist approach:

- In the frequentist view, the parameter θ is **fixed but unknown**.
- In the Bayesian view, the parameter θ is treated as a **random variable**.

The randomness here does not mean that the parameter is *physically fluctuating* — instead, it reflects our **uncertainty** about its value.

6.1.1 Why is this useful?

A **key advantage** of Bayesian inference is that it allows us to **incorporate prior information**¹ about the parameter.

Definition 6.1 (Prior Distribution.). Our uncertainty about θ is represented by a probability distribution, called the **prior distribution**:

$$\theta \sim \pi(\theta).$$

Here $\pi(\theta)$ denotes the prior density (PDF or PMF).

¹**Prior distribution.** In Bayesian inference, the *prior* represents our beliefs about the parameter θ **before seeing any data**. It is written as a probability distribution $\pi(\theta)$, which can be based on previous studies, expert knowledge, or simply chosen to reflect uncertainty (e.g. a flat prior).

Example 6.1. Suppose θ is the average exam score of students in a module. From past experience, the average score tends to be around 60, with most cohorts lying within about 10 marks of this.

We could encode this information as a **Normal prior**:

$$\theta \sim \mathcal{N}(\mu = 60, \sigma^2 = 10^2).$$

This prior expresses our belief *before seeing the new data*.

6.1.2 Python

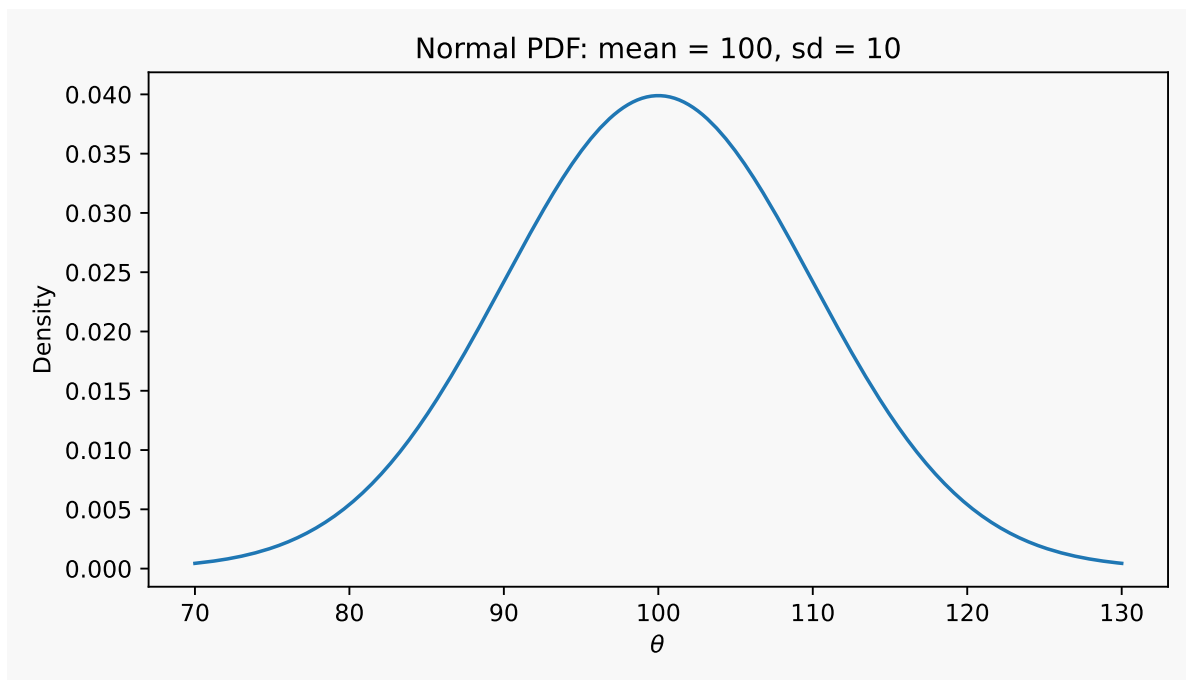
```
import matplotlib.pyplot as plt
import numpy as np
import math

def normal_pdf(theta, mean, sd):
    const = 1 / (sd * (2 * math.pi) ** 0.5)
    exp_term = np.exp(-((theta - mean) / sd) ** 2 / 2)
    return const * exp_term

mean = 100.0
sd = 10.0

theta_grid = np.linspace(mean - 3*sd, mean + 3*sd, 200)
pdf_values = normal_pdf(theta_grid, mean=mean, sd=sd)

plt.figure(figsize=(7,4))
plt.plot(theta_grid, pdf_values)
plt.xlabel(r"$\theta$")
plt.ylabel("Density")
plt.title("Normal PDF: mean = 100, sd = 10")
plt.tight_layout()
```

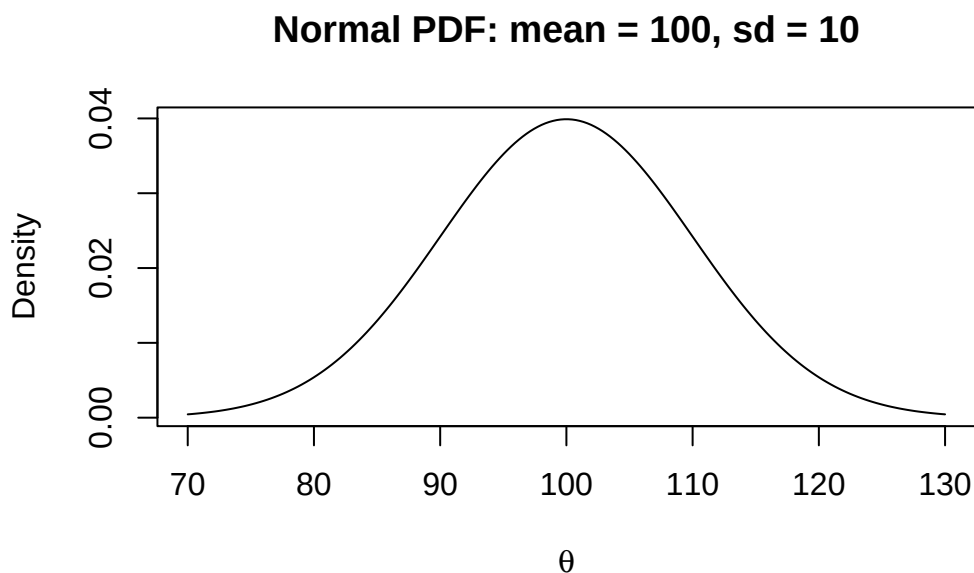


6.1.3 R

```
mean <- 100
sd   <- 10

theta_grid <- seq(mean - 3*sd, mean + 3*sd, length.out = 200)
pdf_values <- dnorm(theta_grid, mean = mean, sd = sd)

plot(theta_grid, pdf_values, type = "l",
      xlab = expression(theta),
      ylab = "Density",
      main = "Normal PDF: mean = 100, sd = 10")
```



6.2 Bayes' Theorem

Bayesian inference takes its name from **Bayes' theorem**, which tells us how to update our beliefs about θ after seeing data.

Recall from probability that for events A and B :

$$\Pr(A \mid B) = \frac{\Pr(B \mid A) \Pr(A)}{\Pr(B)}.$$

To generalise this idea to **random variables and densities**, we use conditional densities:

$$f_{X|Y}(x \mid y) = \frac{f_{XY}(x, y)}{f_Y(y)}.$$

Remark 6.1. Notation. We will usually write $f(x \mid y)$ rather than $f_{X|Y}(x \mid y)$ to keep things simple. Both mean the same thing.

Using this, we obtain Bayes' theorem for densities.

Theorem 6.1 (Bayes' Theorem for Densities.). *For random variables X and Y :*

$$f(x | y) = \frac{f(y | x) f(x)}{f(y)}.$$

Proof. By the definition of conditional density:

$$f(x | y) = \frac{f(x, y)}{f(y)}, \quad f(y | x) = \frac{f(x, y)}{f(x)}.$$

Rearranging the second expression gives $f(x, y) = f(y | x)f(x)$. Substituting this into the first gives

$$f(x | y) = \frac{f(y | x)f(x)}{f(y)}.$$

□

6.3 Bayes' Theorem in Statistics

In Bayesian inference, we apply Bayes' theorem with:

- $x = \theta$ (the parameter, treated as random),
- $y = \underline{x}$ (the observed data).

This gives:

$$\pi(\theta | \underline{x}) = \frac{f(\underline{x} | \theta) \pi(\theta)}{f(\underline{x})}.$$

Definition 6.2 (Posterior Distribution.).

$$\pi(\theta | \underline{x}) = \frac{f(\underline{x} | \theta) \pi(\theta)}{f(\underline{x})}.$$

The **posterior** combines two ingredients:

- The **likelihood** $f(\underline{x} | \theta)$ (information from the data).
- The **prior** $\pi(\theta)$ (information before seeing the data).

The denominator $f(\underline{x})$ is a normalising constant ensuring the posterior integrates to 1.

✦ Big picture:

Bayesian inference updates prior beliefs about parameters using the likelihood from the data, producing the posterior distribution which represents *updated uncertainty*.

Let's start with a simple example to see how this works:

Example 6.2 (Biased Coin). We consider a possibly biased coin with parameter $\theta = \Pr(\text{Head})$. Before seeing data, we express *no preference* over $\theta \in (0, 1)$ via a uniform prior:

$$\theta \sim \text{Uniform}(0, 1) \equiv \text{Beta}(1, 1), \quad \pi(\theta) = 1 \text{ for } 0 < \theta < 1.$$

Hence $E[\theta] = 0.5$.

We toss the coin $n = 5$ times and observe $x = 1$ head. Find the **posterior** $\pi(\theta | x)$.

Solution 6.1. Let X be the number of heads in 5 tosses. Assuming that each coin throw is *independent and identically distributed*, X is Binomial²:

$$X | \theta \sim \text{Bin}(5, \theta).$$

Note that there is only a **single observation** here: one experiment is “tossing the coin 5 times and seeing how many come up heads”. The probability of observing $X = 1$ is

$$f(x = 1 | \theta) = 5\theta(1 - \theta)^4$$

If we plot this:

```
import matplotlib.pyplot as plt
import numpy as np
import math

def binom_lik(theta):
    return 5 * theta * (1 - theta) ** 4

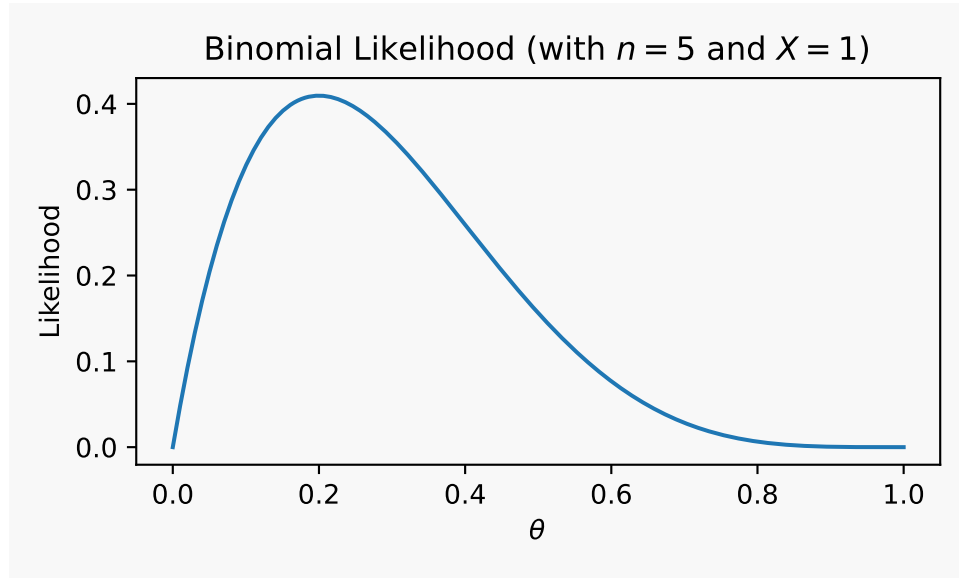
theta_grid = np.linspace(0, 1, 200)
binom_lik_vals = binom_lik(theta_grid)

plt.figure(figsize=(5,3))
```

²**Binomial distribution.** If we assume each coin throw C_i is an i.i.d. Bernoulli(θ) trial with a 1 representing a heads landing, and a 0 representing a tails, the total number of heads out of 5 is $X = C_1 + C_2 + C_3 + C_4 + C_5$. Therefore:

$$X | \theta \sim \text{Bin}(5, \theta).$$


```
plt.plot(theta_grid, binom_lik_vals)
plt.xlabel(r"$\theta$")
plt.ylabel("Likelihood")
plt.title("Binomial Likelihood (with $n=5$ and $X=1$)")
plt.tight_layout()
plt.show()
```



we see that this favours θ around 0.2. In fact, in Example 5.4, we saw that the MLE is $\hat{\theta} = 0.2$.

Our prior for θ was $\pi(\theta) = 1$, for $0 < \theta < 1$. To update our beliefs, by conditioning on the single observation $x = 2$, we use Bayes Theorem 6.2:

$$\pi(\theta|x=1) = \frac{\pi(\theta)f(x=1|\theta)}{f(x=1)} = \frac{1 \times 5\theta(1-\theta)^4}{f(x=1)}, \quad 0 < \theta < 1.$$

To compute the *denominator*, $f(x=1)$, we use the law of total probability 1.3:

$$\begin{aligned} f(x=1) &= \int_{\Theta} \pi(\theta)f(x=1|\theta) d\theta = \int_0^1 1 \times 5\theta(1-\theta)^4 d\theta \\ &= \int_0^1 \theta \times 5(1-\theta)^4 d\theta = [-(1-\theta)^5 \theta]_0^1 + \int_0^1 (1-\theta)^5 d\theta \\ &= 0 + \left[-\frac{(1-\theta)^6}{6} \right]_0^1 = \frac{1}{6}. \end{aligned}$$

Therefore, the posterior density is (for $0 < \theta < 1$):

$$\begin{aligned}\pi(\theta|x=1) &= \frac{\pi(\theta)f(x=1|\theta)}{f(x=1)} = \frac{5\theta(1-\theta)^4}{1/6} \\ &= 30\theta(1-\theta)^4 = \frac{\theta(1-\theta)^4}{B(2,5)}, \quad 0 < \theta < 1,\end{aligned}$$

and so the posterior distribution is $\theta|x=1 \sim \text{Beta}(2,5)$. This distribution has its mode at $\theta = 0.2$, and mean at $E[\theta|x=1] = 2/7 = 0.286$.

The main difficulty in calculating the posterior distribution was in obtaining the $f(x)$ term. However, in many cases we can recognise the posterior distribution without the need to calculate this constant term (constant with respect to θ). In this example, we can calculate the posterior distribution as

$$\begin{aligned}\pi(\theta|x) &\propto \pi(\theta)f(x=1|\theta) \\ &\propto 1 \times 5\theta(1-\theta)^4, \quad 0 < \theta < 1 \\ &= k\theta(1-\theta)^4, \quad 0 < \theta < 1.\end{aligned}$$

As θ is a continuous quantity, what we would like to know is what continuous distribution defined on $(0,1)$ has a probability density function which takes the form $k\theta^{g-1}(1-\theta)^{h-1}$. The answer is the $\text{Beta}(g,h)$ distribution. Therefore, choosing g and h appropriately, we can see that the posterior distribution is $\theta|x=1 \sim \text{Beta}(2,5)$.

Summary:

It is possible that we have a biased coin. If we suppose that all values of $\theta = \text{Pr}(\text{Head})$ are equally likely and then observe 1 head out of 5, then the most likely value of θ is 0.2 — the same as the most likely value from the data alone (not surprising!). However, on average, we would expect θ to be around 0.286. Uncertainty about θ has changed from a (prior) standard deviation of 0.289 to a (posterior) standard deviation of 0.160. The changes in our beliefs about θ are more fully described by the prior and posterior distributions shown in **?@fig-beta-posterior**.

```
// Domain and grid
mDomain = [0, 1]
pdfXs = d3.scaleLinear().domain(mDomain).ticks(200) // 201 points including 0 and 1
```

```

// PDFs (closed-form)
function priorPDF(x)      { return (x > 0 && x < 1) ? 1 : 0; }
function posteriorPDF(x) { return (x > 0 && x < 1) ? 30 * x * (1 - x) ** 4 : 0; }

// Data (use the scalar x from map!)
prior = pdfXs.map(x => ({ x, y: priorPDF(x), which: "Prior  Beta(1,1)" })))
post  = pdfXs.map(x => ({ x, y: posteriorPDF(x), which: "Posterior Beta(2,5)" })))

//| label: fig-beta-posterior
//| fig-cap: Prior and posterior distributions for the coin bias parameter  $\theta = \Pr(\text{text})$ 
//| fig-align: center

bodyFont = getComputedStyle(document.body).fontFamily
// Figure
viewof fig_beta = Plot.plot({
  width: 640,
  height: 320,
  marginLeft: 48,
  marginBottom: 40,
  x: { label: "x", domain: [0, 1] },
  y: { label: "Density" },
  style: { background: "var(--plot-panel-bg)",
    fontSize: 14,
    fontFamily: bodyFont, },
  marks: [
    Plot.ruleY([0], { stroke: "var(--border)", strokeOpacity: 0.6 }),
    Plot.lineY(prior, { x: "x", y: "y", stroke: "var(--brand-red)", strokeWidth: 2, strokeOpacity: 0.5 }),
    Plot.areaY(prior, { x: "x", y: "y", fill: "var(--brand-red)", fillOpacity: 0.05 }),
    Plot.lineY(post, { x: "x", y: "y", stroke: "var(--brand-teal)", strokeWidth: 2 }),
    Plot.areaY(post, { x: "x", y: "y", fill: "var(--brand-teal)", fillOpacity: 0.2 }),
    Plot.text(
      [
        { x: 0.8, y: priorPDF(0.8), label: "Prior" },
        { x: 0.35, y: posteriorPDF(0.3), label: "Posterior" }
      ],
      { x: "x", y: "y", text: "label", dy: -8 }
    )
  ]
})

```

Example 6.3. Consider an experiment to determine how good a music expert is at distinguishing between pages from Haydn and Mozart scores. Let $\theta = \Pr(\text{correct choice})$. Suppose that, before conducting the experiment, we have been told that the expert is very competent. In fact, it is suggested that we should have a prior distribution which has a mode around $\theta = 0.95$ and for which $\Pr(\theta < 0.8)$ is very small. We choose $\theta \sim \text{Beta}(77, 5)$, with probability density function

$$\pi(\theta) = 128107980 \theta^{76} (1 - \theta)^4, \quad 0 < \theta < 1.$$

A graph of this prior density is given as follows:

6.3.1 Python

```
import numpy as np
import math
import matplotlib.pyplot as plt

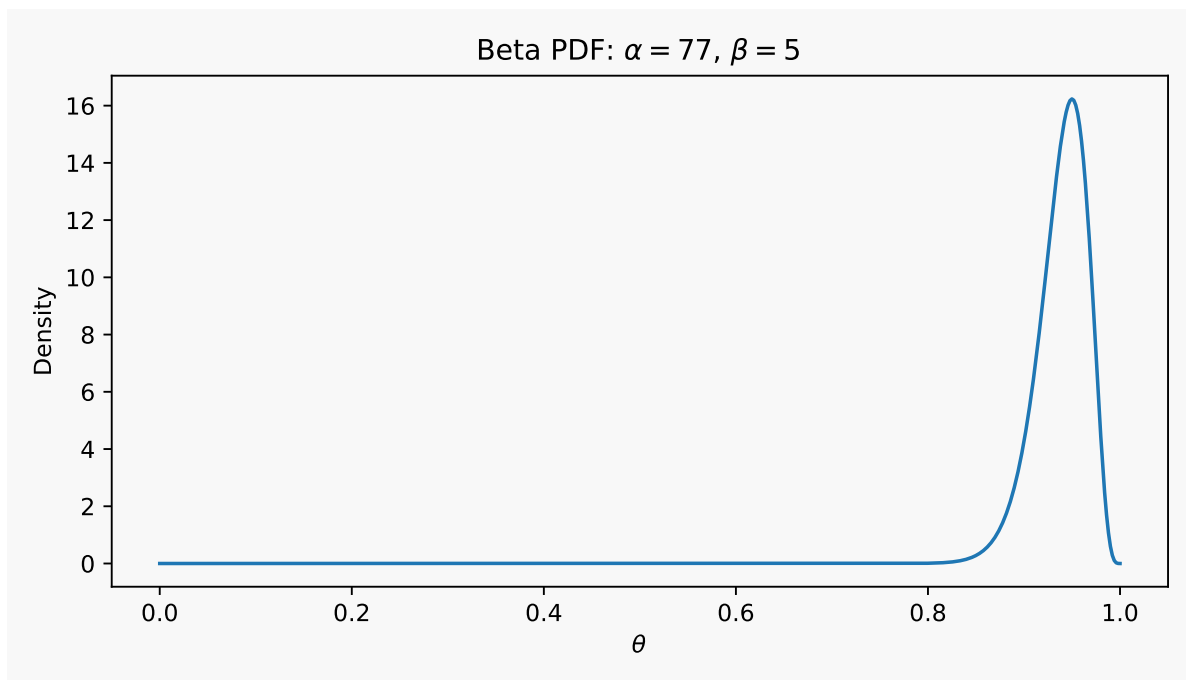
# Beta( , ) with =77, =5 → ( ) ^ (76) (1- ) ^ (4)
alpha, beta = 77, 5

# B( , ) = Γ( )Γ( )/Γ( + )    1/B = exp(lgamma( + ) - lgamma( ) - lgamma( ))
# Note the lgamma function computes the logarithm of the Gamma function.
beta_const_inv = math.exp(math.lgamma(alpha + beta) - math.lgamma(alpha) - math.lgamma(beta))

def beta_pdf(theta):
    return beta_const_inv * np.power(theta, alpha - 1) * np.power(1 - theta, beta - 1)

theta_grid = np.linspace(0.0, 1.0, 500)
pdf_values = beta_pdf(theta_grid)

plt.figure(figsize=(7,4))
plt.plot(theta_grid, pdf_values)
plt.xlabel(r"$\theta$")
plt.ylabel("Density")
plt.title(rf"Beta PDF: $\alpha$={alpha}$, $\beta$={beta}$")
# plt.xlim(0, 1)
plt.tight_layout()
plt.show()
```



6.3.2 R

```
alpha <- 77
beta  <- 5

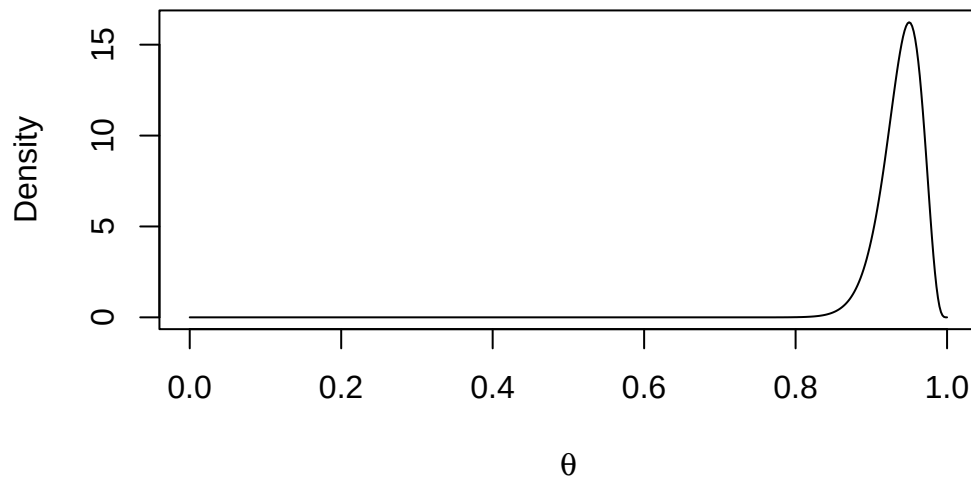
# Normalising constant via log-gamma (works like Python's lgamma)
beta_const_inv <- exp(lgamma(alpha + beta) - lgamma(alpha) - lgamma(beta))

beta_pdf <- function(theta) {
  beta_const_inv * theta^(alpha - 1) * (1 - theta)^(beta - 1)
}

theta_grid <- seq(0, 1, length.out = 500)
pdf_values <- beta_pdf(theta_grid)

plot(theta_grid, pdf_values, type = "l",
      xlab = expression(theta), ylab = "Density",
      main = bquote("Beta PDF: " ~ alpha == .(alpha) ~ ", " ~ beta == .(beta)))
```

Beta PDF: $\alpha = 77$, $\beta = 5$



6.4 Bayesian vs. Frequentist (again)

Part III

Part III - Interval Estimation

7 Frequentist Interval Estimation

Recalling the main inference tasks 3.2, we now move from *point estimation* to *interval estimation*.

The key idea is that instead of giving a single “best guess”¹ of a parameter, we want to provide a **range of plausible values**. In other words, an **interval**.

7.1 The Frequentist Perspective

Recall that, for frequentists, the parameter θ is **fixed but unknown**.

The randomness comes from the data: if we repeated the experiment many times, we would get different samples, different estimates, and hence different intervals.

- **Point Estimate:** a single statistic $\hat{\theta}(\underline{X})$.
- **Interval estimate:** two statistics $L(\underline{X})$ and $U(\underline{X})$ that form a *random interval*.

Definition 7.1 (Confidence Interval). A $100(1 - \alpha)\%$ **confidence interval** for θ is a *random interval*

$$[L(\underline{X}), U(\underline{X})]$$

satisfying

$$\Pr(L(\underline{X}) < \theta < U(\underline{X})) = 1 - \alpha.$$

This property is called **coverage**: in the long run, a proportion $1 - \alpha$ of such intervals (constructed from repeated experiments) will contain the true θ .

7.2 One Observed Sample

In practice, we only ever observe one dataset $\underline{x}$. So we calculate the observed interval:

$$[L_{\text{obs}}, U_{\text{obs}}] = [L(\underline{x}), U(\underline{x})].$$

¹Recall from Chapter 6 that Bayesians do not return a single “best guess,” but a full **posterior distribution** over parameter values. From this distribution we can compute **summaries** such as plausible ranges of θ (see Chapter 8), which play a role similar to confidence intervals but with a different interpretation.

! Confidence Interval Interpretation

Once the data are observed, this interval is **fixed, not random**.

Therefore, a frequentist would **not** say

$$\Pr(L_{\text{obs}} < \theta < U_{\text{obs}}) = 1 - \alpha.$$

That statement is meaningless, because θ is not random in the frequentist view.

Instead, the correct interpretation is: **our method has long-run coverage $1 - \alpha$** .

7.3 Deriving Confidence Intervals

Exact confidence intervals are not always available. When possible, we aim for **well-calibrated** (exact coverage) intervals; otherwise, we use **approximate intervals** (e.g. via the Central Limit Theorem).

However, there are a few cases where we can compute closed-form **well-calibrated** confidence intervals:

7.3.1 The Normal case (Known Variance)

Suppose we observe n i.i.d. samples from a normal distribution with known variance:

$$X_i \sim \mathcal{N}(\mu, \sigma^2), \quad i = 1, \dots, n.$$

- The sample mean $\bar{X}$ has distribution:

$$\bar{X} \sim \mathcal{N}\left(\mu, \frac{\sigma^2}{n}\right).$$

- Standardising:

$$Z = \frac{\bar{X} - \mu}{\sigma/\sqrt{n}} \sim \mathcal{N}(0, 1).$$

- From quantiles of the standard normal, we obtain the $(1 - \alpha)$ confidence interval:

$$\bar{X} \pm z_{1-\alpha/2} \frac{\sigma}{\sqrt{n}}.$$

7.3.2 Normal Case (Unknown Variance)

If σ^2 is unknown, replace it by the sample variance S^2 . Then

$$T = \frac{\bar{X} - \mu}{S/\sqrt{n}} \sim t_{n-1}.$$

So the $(1 - \alpha)$ confidence interval becomes:

$$\bar{X} \pm t_{n-1, 1-\alpha/2} \frac{S}{\sqrt{n}}.$$

7.3.3 Approximate Intervals via the CLT

For many distributions (not just normal), the **Central Limit Theorem (CLT)** tells us that

$$\frac{\hat{\theta} - \theta}{\text{SE}(\hat{\theta})} \approx \mathcal{N}(0, 1)$$

for large n , where $\text{SE}(\hat{\theta})$ is the standard error.

Thus, a general approximate confidence interval is:

$$\hat{\theta} \pm z_{1-\alpha/2} \text{SE}(\hat{\theta}).$$

8 Bayesian Intervals

8.1 Bayesian Credible Intervals

Part IV

Part IV — Hypothesis Testing

9 Frequentist Hypothesis Testing

Recalling the main inference tasks 3.2, we now move from *interval estimation* to *hypothesis testing*.

The name **hypothesis testing** comes from the **scientific method**:

1. Propose a hypothesis about the world.
2. Collect data through experiment or observation.
3. Assess whether the data are consistent with the hypothesis.

If the data are consistent, this provides **support** for the hypothesis (though never proof). If the data look very unlikely under the hypothesis, this is evidence *against* it.

9.1 Formal Setup

Unlike estimation (which gives a range of plausible values), hypothesis testing asks a **yes/no question** about the parameter θ .

We formalise this by specifying a **null hypothesis** H_0 , a concrete *mathematical statement* about θ :

$$H_0 : \theta \in \Theta_0,$$

where $\Theta_0 \subseteq \Theta$ is part of the parameter space.

We then use the observed data $\underline{x}$ to evaluate whether H_0 is plausible. The frequentist reasoning is:

1. **Assume H_0 is true.**
2. Use the statistical model to work out what kinds of data would be typical under H_0 .
3. Compare the actual data with this prediction. If the data look very unusual under H_0 , this counts against the hypothesis.

Visualisation

Suppose our model is

$$X_i \mid \gamma \sim \mathcal{N}(200, \gamma),$$

where γ is the variance.

We want to test whether $\gamma = 10$. That is:

$$H_0 : \gamma = 10.$$

The frequentist approach is: *assume H_0 is true* and then check whether the observed data are typical of a normal distribution with variance 10.

```
// Parameters
n = 30
mu0 = 200      // null mean
sigma = 10     // std deviation under null
alpha = 0.05

// Slider for observed sample mean
viewof xbar_obs = Inputs.range([mu0 - 3*sigma, mu0 + 3*sigma],
  {step: 0.5, value: mu0, label: md`Observed sample mean ${tex`\bar{x}_{obs}`}` } )

// Null sampling distribution
xgrid = d3.range(mu0 - 4*sigma, mu0 + 4*sigma, 0.1)
pdf = xgrid.map(x => ({x, y: d3.randomNormal(mu0, sigma/Math.sqrt(n))() }))) // or use normal

// Compute rejection region bounds
zcrit = d3.quantile(d3.range(-5,5,0.01), alpha/2) // could just hardcode 1.96
lower = mu0 - 1.96 * sigma/Math.sqrt(n)
upper = mu0 + 1.96 * sigma/Math.sqrt(n)

// Plot
Plot.plot({
  width: 600, height: 300,
  x: {label: "Sample mean"},
  y: {label: "Density"},
  marks: [
    // Null distribution
    Plot.line(xgrid.map(x => ({x, y: 1/(sigma/Math.sqrt(n)*Math.sqrt(2*Math.PI))
```

```

                                * Math.exp(-0.5*((x-mu0)/(sigma/Math.sqrt(n))**2))),
    {x:"x", y:"y", stroke: "var(--brand-red)" }},

// Shaded rejection regions
Plot.areaY(xgrid.map(x => ({x, y: (x<lower||x>upper) ?
    1/(sigma/Math.sqrt(n)*Math.sqrt(2*Math.PI))
    * Math.exp(-0.5*((x-mu0)/(sigma/Math.sqrt(n))**2) : 0})),
    {x:"x", y:"y", fill: "var(--brand-red)", fillOpacity:0.3}),

// Observed value
Plot.ruleX([xbar_obs], {stroke:"black", strokeWidth:2}),
]
})

```

9.2 Test Statistics and the Testing Procedure

Working directly with the full data distribution can be complicated. Instead, we reduce the data to a **test statistic** $T(\underline{X})$ that captures the evidence relevant to H_0 .

The general frequentist procedure is:

1. **Choose a test statistic** $T(\underline{X})$ that is sensitive to departures from H_0 .
2. **Derive its sampling distribution under** H_0 .
3. **Compute the observed test statistic** T_{obs} from the data.
4. **Calculate a p -value:** the probability of observing $T(\underline{X})$ at least as extreme as T_{obs} , assuming H_0 is true.
5. **Decision:**
 - If the p -value is below a chosen threshold α (the **significance level**), we **reject** H_0 .
 - Otherwise, we **fail to reject** H_0 ¹.

¹It is common to say “reject H_0 ” when the p -value is small, and “fail to reject H_0 ” otherwise. Both phrases should be interpreted with care: rejecting H_0 does **not** prove it is false, and failing to reject H_0 does **not** prove it is true. Instead, hypothesis testing provides a decision rule about whether the data are sufficiently inconsistent with H_0 at a chosen significance level.

9.3 Types of Errors

Because decisions are based on data (which vary by chance), mistakes are inevitable:

- **Type I Error:** rejecting H_0 when it is actually true.
- **Type II Error:** failing to reject H_0 when the alternative is true.

We will return to these concepts later when discussing test performance and power.

10 Bayes Factors

Part V

Additional Material

Formula Sheet

Discrete distributions

Bernoulli

Model. $X \sim \text{Bernoulli}(p)$ with $0 < p < 1$.

Support. $x \in \{0, 1\}$.

PMF.

$$\Pr(X = x) = p^x(1 - p)^{1-x}, \quad x \in \{0, 1\}.$$

Mean/Var. $\mathbb{E}[X] = p, \quad \text{Var}(X) = p(1 - p).$

Binomial

Model. $X \sim \text{Bin}(n, p), n \in \mathbb{N}, 0 < p < 1$.

Support. $x = 0, 1, \dots, n$.

PMF.

$$\Pr(X = x) = \binom{n}{x} p^x (1 - p)^{n-x}.$$

Mean/Var. $\mathbb{E}[X] = np, \quad \text{Var}(X) = np(1 - p).$

Poisson

Model. $X \sim \text{Po}(\lambda), \lambda > 0$.

Support. $x = 0, 1, 2, \dots$.

PMF.

$$\Pr(X = x) = \frac{\lambda^x e^{-\lambda}}{x!}.$$

Mean/Var. $\mathbb{E}[X] = \lambda, \quad \text{Var}(X) = \lambda.$

Geometric

Model. $X \sim \text{Geom}(p)$ (trials **until first success**, counting the success), $0 < p < 1$.

Support. $x = 1, 2, \dots$

PMF.

$$\Pr(X = x) = p(1 - p)^{x-1}.$$

Mean/Var. $\mathbb{E}[X] = \frac{1}{p}$, $\text{Var}(X) = \frac{1-p}{p^2}$.



Tip

Alternative convention. Some texts start at $x = 0$ (failures before the first success). Adjust mean/variance accordingly.

Negative Binomial

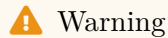
Model. $X \sim \text{NegBin}(r, p)$: number of trials to achieve the r -th success (counts the successes), $r > 0$ (often integer), $0 < p < 1$.

Support. $x = r, r + 1, \dots$

PMF.

$$\Pr(X = x) = \binom{x-1}{r-1} p^r (1-p)^{x-r}.$$

Mean/Var. $\mathbb{E}[X] = \frac{r}{p}$, $\text{Var}(X) = \frac{r(1-p)}{p^2}$.



Warning

Parameterisation alert. Another common version counts **failures** before the r -th success ($x = 0, 1, \dots$) with different mean/variance. Always check which one is used.

Continuous distributions

Uniform (continuous)

Model. $X \sim \text{Unif}(a, b)$, $a < b$.

Support. $a < x < b$.

PDF.

$$f_X(x; a, b) = \frac{1}{b - a}.$$

Mean/Var. $\mathbb{E}[X] = \frac{a+b}{2}$, $\text{Var}(X) = \frac{(b-a)^2}{12}$.

Exponential

Model. $X \sim \text{Exp}(\lambda)$ with **rate** $\lambda > 0$.

Support. $x > 0$.

PDF.

$$f_X(x; \lambda) = \lambda e^{-\lambda x}.$$

Mean/Var. $\mathbb{E}[X] = \frac{1}{\lambda}$, $\text{Var}(X) = \frac{1}{\lambda^2}$.

Gamma

Model. $X \sim \text{Gamma}(a, b)$ with **shape** $a > 0$ and **rate** $b > 0$.

Support. $x > 0$.

PDF.

$$f_X(x; a, b) = \frac{b^a}{\Gamma(a)} x^{a-1} e^{-bx}.$$

Mean/Var. $\mathbb{E}[X] = \frac{a}{b}$, $\text{Var}(X) = \frac{a}{b^2}$.



Tip

Many sources use **scale** $\beta = 1/b$: then $f(x) = \frac{1}{\Gamma(a)\beta^a} x^{a-1} e^{-x/\beta}$ and $\mathbb{E}[X] = a\beta$.

Normal

Model. $X \sim \mathcal{N}(\mu, \sigma^2)$ with $\sigma > 0$.

Support. $x \in \mathbb{R}$.

PDF.

$$f_X(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

Mean/Var. $\mathbb{E}[X] = \mu$, $\text{Var}(X) = \sigma^2$.

Lognormal

Model. $X \sim \text{LogNormal}(\mu, \sigma^2)$ meaning $\ln X \sim \mathcal{N}(\mu, \sigma^2)$.

Support. $x > 0$.

PDF.

$$f_X(x; \mu, \sigma^2) = \frac{1}{x\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right).$$

Mean/Var. $\mathbb{E}[X] = e^{\mu + \sigma^2/2}$, $\text{Var}(X) = (e^{\sigma^2} - 1)e^{2\mu + \sigma^2}$.

Chi-squared

Model. $X \sim \chi_\nu^2$ with $\nu > 0$ degrees of freedom.

Support. $x > 0$.

PDF.

$$f_X(x; \nu) = \frac{1}{2^{\nu/2} \Gamma(\nu/2)} x^{\nu/2-1} e^{-x/2}.$$

Mean/Var. $\mathbb{E}[X] = \nu$, $\text{Var}(X) = 2\nu$.

i Note

χ_ν^2 is a special case of Gamma with $a = \nu/2$, $b = 1/2$.

Student- t

Model. $X \sim t_\nu$ with $\nu > 0$ degrees of freedom.

Support. $x \in \mathbb{R}$.

PDF.

$$f_X(x; \nu) = \frac{\Gamma(\frac{\nu+1}{2})}{\sqrt{\nu\pi} \Gamma(\frac{\nu}{2})} \left(1 + \frac{x^2}{\nu}\right)^{-(\nu+1)/2}.$$

Mean/Var. $\mathbb{E}[X] = 0$ for $\nu > 1$; undefined for $\nu \leq 1$.

$\text{Var}(X) = \frac{\nu}{\nu-2}$ for $\nu > 2$; infinite for $1 < \nu \leq 2$; undefined for $\nu \leq 1$.

F distribution

Model. $X \sim F_{d_1, d_2}$ with $d_1, d_2 > 0$ degrees of freedom.

Support. $x > 0$.

PDF.

$$f_X(x; d_1, d_2) = \frac{\Gamma(\frac{d_1+d_2}{2})}{\Gamma(\frac{d_1}{2}) \Gamma(\frac{d_2}{2})} \left(\frac{d_1}{d_2}\right)^{d_1/2} \frac{x^{d_1/2-1}}{\left(1 + \frac{d_1}{d_2}x\right)^{(d_1+d_2)/2}}.$$

Mean/Var. $\mathbb{E}[X] = \frac{d_2}{d_2-2}$ for $d_2 > 2$.

$\text{Var}(X) = \frac{2d_2^2(d_1+d_2-2)}{d_1(d_2-2)^2(d_2-4)}$ for $d_2 > 4$.

Beta

Model. $X \sim \text{Beta}(a, b)$, $a > 0, b > 0$.

Support. $0 < x < 1$.

PDF.

$$f_X(x; a, b) = \frac{\Gamma(a+b)}{\Gamma(a)\Gamma(b)} x^{a-1}(1-x)^{b-1}.$$

Mean/Var. $\mathbb{E}[X] = \frac{a}{a+b}$, $\text{Var}(X) = \frac{ab}{(a+b)^2(a+b+1)}$.

Multivariate Normal

Model. $\mathbf{X} \sim \mathcal{N}_p(\mu, \Sigma)$ with Σ positive definite.

Support. $\mathbf{x} \in \mathbb{R}^p$.

PDF.

$$f_{\mathbf{X}}(\mathbf{x}; \mu, \Sigma) = \frac{1}{(2\pi)^{p/2}(\det \Sigma)^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \mu)^\top \Sigma^{-1}(\mathbf{x} - \mu)\right).$$

Mean/Cov. $\mathbb{E}[\mathbf{X}] = \mu$, $\text{Var}(\mathbf{X}) = \Sigma$.

Integral reminder

The Gamma function:

$$\Gamma(a) = \int_0^\infty x^{a-1} e^{-x} dx, \quad a > 0.$$

! Important

Common pitfalls. - **Rate vs scale** for Exponential/Gamma: check whether the parameter is λ (rate) or β (scale). - **Geometric/Negative Binomial** supports vary by convention (counting failures vs trials). Always verify support and mean/variance before using formulas. - In likelihood work, write $f_X(x; \theta)$ (semicolon) to emphasise **data vs parameters**.

Revision Sheet

Under Construction...

Distribution Explorer

```
// Global helpers (no theme logic needed)
ACCENT  = "var(--brand-teal)"
ACCENT2 = "var(--brand-red)"
GRID    = "var(--border)"

// Numerical helpers
linspace = (a, b, n=401) => Array.from({length:n}, (_,i)=> a + (b-a)*(i/(n-1)))

// Log-Gamma (Lanczos) and friends (for stable PMFs/PDFs)
function logGamma(z){
  const g = 7;
  const p = [
    0.99999999999980993, 676.5203681218851, -1259.1392167224028,
    771.32342877765313, -176.61502916214059, 12.507343278686905,
    -0.13857109526572012, 9.9843695780195716e-6, 1.5056327351493116e-7
  ];
  if(z < 0.5){
    return Math.log(Math.PI) - Math.log(Math.sin(Math.PI*z)) - logGamma(1 - z);
  }
  z -= 1;
  let x = p[0];
  for(let i=1; i<p.length; i++) x += p[i] / (z + i);
  const t = z + g + 0.5;
  return 0.5*Math.log(2*Math.PI) + (z+0.5)*Math.log(t) - t + Math.log(x);
}

logChoose = (n,k) => logGamma(n+1) - logGamma(k+1) - logGamma(n-k+1)

// Discrete PMFs (stable in log-domain)
binomPMF = (n,p,k) => Math.exp(logChoose(n,k) + k*Math.log(p) + (n-k)*Math.log(1-p))
poisPMF  = (lambda,k) => Math.exp(k*Math.log(lambda) - lambda - logGamma(k+1))
geomPMF1 = (p,k) => p * Math.pow(1-p, k-1) // support {1,2,...}
negbinPMF = (r,p,x) => {
  if(x < r) return 0;
```

```

    return Math.exp(logChoose(x-1, r-1) + r*Math.log(p) + (x-r)*Math.log(1-p));
}

// Continuous PDFs
normPDF = (x,mu,sig) => Math.exp(-0.5*((x-mu)/sig)**2)/(sig*Math.sqrt(2*Math.PI))
lognormPDF = (x,mu,sig) => x<=0 ? 0 : Math.exp(-((Math.log(x)-mu)**2)/(2*sig*sig)) / (x*sig)
unifPDF = (x,a,b) => (x>a && x<b) ? 1/(b-a) : 0
expPDF = (x,lambda) => x>0 ? lambda*Math.exp(-lambda*x) : 0
gammaPDF = (x,a,b) => { // shape a, rate b
    if(x<=0) return 0;
    return Math.exp(a*Math.log(b) - logGamma(a) + (a-1)*Math.log(x) - b*x)
}
betaPDF = (x,a,b) => {
    if(x<=0 || x>=1) return 0;
    const logB = logGamma(a)+logGamma(b)-logGamma(a+b);
    return Math.exp((a-1)*Math.log(x) + (b-1)*Math.log(1-x) - logB)
}
chisqPDF = (x,nu) => gammaPDF(x, nu/2, 1/2)
tPDF = (x,nu) => {
    const logC = logGamma((nu+1)/2) - (0.5*Math.log(nu*Math.PI) + logGamma(nu/2));
    return Math.exp(logC - ((nu+1)/2)*Math.log(1 + (x*x)/nu));
}
fPDF = (x,d1,d2) => {
    if(x<=0) return 0;
    const logC = logGamma((d1+d2)/2) - (logGamma(d1/2)+logGamma(d2/2)) + (d1/2)*Math.log(d1/d2)
    return Math.exp(logC + (d1/2 - 1)*Math.log(x) - ((d1+d2)/2)*Math.log(1 + (d1/d2)*x))
}

// Convenient domain heuristics
domainNormal = (mu,sig) => [mu - 4*sig, mu + 4*sig]
domainGamma = (a,b) => [0, Math.max(5, a/b + 6*Math.sqrt(a)/b)]
domainExp = (lambda) => [0, Math.max(5, 6/lambda)]
domainBeta = [0,1]
domainPos = [0, 10]

```

10.1 Bernoulli

```

{
    const p = bern_p;
    const data = [0, 1].map(x => ({ x, y: x ? p : 1 - p }));
}

```

```

const stats = md`**Mean**  $p$  =  $\{p.toFixed(2)\}$ ;
**Var**  $p(1-p)$  =  $\{(p * (1 - p)).toFixed(3)\}$ .`;

const plot = Plot.plot({
  width, height: 260,
  style: {
    background: "var(--plot-panel-bg)",
    color: "var(--brand-fg)" // drives 'currentColor'
  },

  y: { label: "PMF", domain: [0, 1] },
  x: { label: "x", type: "band", domain: [0, 1] }, // <-- band scale
  marks: [
    Plot.barY(data, { x: "x", y: "y", fill: ACCENT }),
    Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
  ]
});

return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

```

```

viewof bern_p = Inputs.range([0.01, 0.99], {
  value: 0.6, step: 0.01, label: md` $p$ `
})

```

10.2 Binomial

```

{
  const n = binom_n, p = binom_p;
  const data = Array.from({ length: n + 1 }, (_, k) => ({ x: k, y: binomPMF(n, p, k) }));
  const xDomain = Array.from({ length: n + 1 }, (_, k) => k); // <-- 0..n categories

  const stats = md`**Mean**  $np$  =  $\{(n * p).toFixed(2)\}$ ;
  **Var**  $np(1-p)$  =  $\{(n * p * (1 - p)).toFixed(2)\}$ .`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",

```

```

    color: "var(--brand-fg)"          // drives 'currentColor'
  },
  y: { label: "PMF" },
  x: { label: "x", type: "band", domain: xDomain, padding: 0.05 }, // <-- band scale
  marks: [
    Plot.barY(data, { x: "x", y: "y", fill: ACCENT }),
    Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
  ]
});

return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

```

```

viewof binom_n = Inputs.range([1, 200], { value: 20, step: 1, label: md`${tex`n`}` });
viewof binom_p = Inputs.range([0.01, 0.99], { value: 0.3, step: 0.01, label: md`${tex`p`}` });

```

10.3 Poisson

```

{
  // compute locally (no exported names)
  const L      = pois_lambda;
  const kmax   = Math.max(15, Math.ceil(L + 6*Math.sqrt(L)));
  const pois_data = Array.from({length: kmax + 1}, (_, k) => ({ x: k, y: poisPMF(L, k) }));

  // build the UI bits
  const stats = md`**Mean** ${tex`= \lambda` } = ${L.toFixed(2)};
  **Var** ${tex`= \lambda` } = ${L.toFixed(2)}.`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)"          // drives 'currentColor'
    },
    y: { label: "PMF" },
    x: { label: "x" },
    marks: [
      Plot.barY(pois_data, { x: "x", y: "y", fill: "var(--brand-teal)" }),

```

```

    Plot.ruleY([0], { stroke: "var(--border)", strokeOpacity: 0.6 })
  ]
});

// return both in a single wrapper
return html`<div class="dist-panel">
  <div class="stats">${stats}</div>
  ${plot}
</div>`;
}

```

```

viewof pois_lambda = Inputs.range([0.2, 30], {
  value: 6, step: 0.2, label: md`${tex`\lambda`}`
})

```

10.4 Geometric

```

{
  // compute locally
  const p = geom_p;
  const raw = Math.log(1e-4) / Math.log(1 - p); // tail cutoff
  const kmax = Math.max(5, Math.min(200, Math.ceil(raw))); // clamp for stability
  const geom_data = Array.from({ length: kmax }, (_, i) => {
    const k = i + 1;
    return { x: k, y: geomPMF1(p, k) }; // PMF = p(1-p)^{k-1}
  });

  // stats readout
  const stats = md`**Mean** ${tex`= 1/p`} = ${(1/p).toFixed(2)};
  **Var** ${tex`= (1-p)/p^2`} = ${(((1-p)/(p*p))).toFixed(2)}.`;

  // plot
  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
    y: { label: "PMF" },

```

```

    x: { label: "x" },
    marks: [
      Plot.barY(geom_data, { x: "x", y: "y", fill: ACCENT }),
      Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
    ]
  });

  // return one wrapper node
  return html`<div class="dist-panel">
    <div class="stats">${stats}</div>
    ${plot}
  </div>`;
}

```

```

viewof geom_p = Inputs.range([0.02, 0.98], {
  value: 0.3, step: 0.02, label: md`${tex`p`}`
})

```

10.5 Negative Binomial

```

{
  const r = Math.round(negbin_r), p = negbin_p;
  const mean = r / p, sd = Math.sqrt(r * (1 - p)) / p;
  const xmax = Math.min(400, Math.ceil(mean + 6 * sd));

  const data = Array.from({ length: Math.max(0, xmax - r + 1) }, (_, i) => {
    const x = r + i;
    return { x, y: negbinPMF(r, p, x) };
  });

  const stats = md`**Mean** ${tex`= r/p`} = ${ (r/p).toFixed(2) };
  **Var** ${tex`= r(1-p)/p^2`} = ${ ((r*(1-p))/(p**2)).toFixed(2) }.`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
  },

```

```

y: { label: "PMF" }, x: { label: "x" },
  marks: [
    Plot.barY(data, { x: "x", y: "y", fill: ACCENT }),
    Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
  ]
});

return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

```

```

viewof negbin_r = Inputs.range([1, 30], {value: 5, step: 1, label: md`${tex`r`}`})
viewof negbin_p = Inputs.range([0.02, 0.98], {value: 0.4, step: 0.02, label: md`${tex`p`}`})

```

10.6 Uniform

```

{
  const a = unif_a, b = unif_b;
  const data = linspace(a - 1, b + 1, 401).map(x => ({ x, y: unifPDF(x, a, b) }));

  const stats = md`**Mean** ${tex`= (a+b)/2`} = ${(((a+b)/2)).toFixed(2)};
  **Var** ${tex`= (b-a)^2/12`} = ${(((b-a)**2)/12).toFixed(3)}.`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
    y: { label: "PDF" }, x: { label: "x" },
    marks: [
      Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),
      Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),
      Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
    ]
  });

  return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

```

```
viewof unif_a = Inputs.range([-5, 5], {value: -2, step: 0.1, label: md`${tex`a`}`})
viewof unif_b = Inputs.range([unif_a + 0.2, unif_a + 10], {value: 3, step: 0.1, label: md`${
```

10.7 Exponential

```
{
  const    = exp_lambda;
  const [lo, hi] = domainExp();
  const data = linspace(lo, hi).map(x => ({ x, y: expPDF(x, ) }));

  const stats = md`**Mean** ${tex`= 1/\lambda`} = ${(1/).toFixed(2)};
  **Var** ${tex`= 1/\lambda^2`} = ${(1/(**2)).toFixed(2)}.`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
    y: { label: "PDF" }, x: { label: "x" },
    marks: [
      Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),
      Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),
      Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
    ]
  });

  return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}
```

```
viewof exp_lambda = Inputs.range([0.2, 5], {value: 1, step: 0.1, label: md`${tex`\lambda`}`})
```

10.8 Gamma

```
{
  const a = gamma_a, b = gamma_b;
  const [lo, hi] = domainGamma(a, b);
```



```

const data = linspace(lo, hi).map(x => ({ x, y: gammaPDF(x, a, b) }));

const stats = md`**Mean**  $\text{a/b}$  =  $\{(a/b).toFixed(2)\}$ ;
**Var**  $\text{a/b}^2$  =  $\{(a/(b**2)).toFixed(2)\}$ .`;

const plot = Plot.plot({
  width, height: 260,
  style: {
    background: "var(--plot-panel-bg)",
    color: "var(--brand-fg)" // drives 'currentColor'
  },
  y: { label: "PDF" }, x: { label: "x" },
  marks: [
    Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),
    Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),
    Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
  ]
});

return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

```

```

viewof gamma_a = Inputs.range([0.5, 10], {value: 3, step: 0.1, label: md` $a$  \text{(shape)}`});
viewof gamma_b = Inputs.range([0.2, 5], {value: 1, step: 0.1, label: md` $b$  \text{(rate)}`});

```

10.9 Normal

```

{
  const mu = norm_mu, sig = norm_sig;
  const [lo, hi] = domainNormal(mu, sig);
  const data = linspace(lo, hi).map(x => ({ x, y: normPDF(x, mu, sig) }));

  const stats = md`**Mean**  $\mu$  =  $\{.toFixed(2)\}$ ;
  **Var**  $\sigma^2$  =  $\{(sig**2).toFixed(2)\}$ .`;

  const plot = Plot.plot({
    width, height: 260,
    style: { background: "var(--plot-panel-bg)", color: "var(--brand-fg)" },
    x: { label: "x" }, y: { label: "PDF" },
    marks: [

```

```

    // shaded area under the PDF:
    Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),

    // PDF outline:
    Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),

    Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
  ]
});

return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

viewof norm_mu = Inputs.range([-3, 3], {value: 0, step: 0.05, label: md`${tex`\mu`}`})
viewof norm_sig = Inputs.range([0.05, 3], {value: 1, step: 0.05, label: md`${tex`\sigma`}`})

```

10.10 Lognormal

```

{
  const [lnorm_mu, lnorm_sig] = [norm_mu, norm_sig];
  const hi = Math.exp( + 5 * lnorm_sig );
  const data = linspace(0, hi).map(x => ({ x, y: lognormPDF(x, lnorm_mu, lnorm_sig) }));

  const stats = md`**Mean**  $e^{\mu + \sigma^2/2}$  =  $(\text{Math.exp}(\mu + (\sigma^2)/2)).toFixed(2)$ 
  **Var**  $(e^{\sigma^2}-1)e^{2\mu + \sigma^2}$  =  $((\text{Math.exp}(\sigma^2)-1)*\text{Math.exp}(2*\mu + \sigma^2)).toFixed(2)$ `;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
    y: { label: "PDF" }, x: { label: "x" },
    marks: [
      Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),
      Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),
      Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
    ]
  });
}

```

```

    return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
  }

```

```

viewof lnorm_mu = Inputs.range([-1.5, 2], {value: 0, step: 0.1, label: md`${tex`\mu` (\log
viewof lnorm_sig = Inputs.range([0.01, 1.5], {value: 0.6, step: 0.01, label: md`${tex`\sigma`

```

10.11 Chi-squared

```

{
  const    = Math.round(chi_nu);
  const hi = Math.max(5,    + 6 * Math.sqrt(2 * ));
  const data = linspace(0, hi).map(x => ({ x, y: chisqPDF(x, ) }));

  const stats = md`**Mean** ${tex`= \nu`} = ${ };
  **Var** ${tex`= 2\nu`} = ${2*}.`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
    y: { label: "PDF" }, x: { label: "x" },
    marks: [
      Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),
      Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),
      Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
    ]
  });

  return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}

```

```

viewof chi_nu = Inputs.range([1, 40], {value: 8, step: 1, label: md`${tex`\nu`}})

```

10.12 Student t

```
{
  const t_nu = Math.round(t_nu);
  const data = linspace(-6, 6).map(x => ({ x, y: tPDF(x, t_nu) }));

  const stats = md`${tex`\mathrm{E}[X]=0 (\nu>1)`},
    `${tex`\operatorname{Var}(X)=\nu/(\nu-2)\ (\nu>2)`}.
  Current: ${t_nu}.`;

  const plot = Plot.plot({
    width, height: 260,
    style: {
      background: "var(--plot-panel-bg)",
      color: "var(--brand-fg)" // drives 'currentColor'
    },
    y: { label: "PDF" }, x: { label: "x" },
    marks: [
      Plot.lineY(data, { x: "x", y: "y", stroke: ACCENT, strokeWidth: 2 }),
      Plot.areaY(data, { x: "x", y: "y", fill: ACCENT, fillOpacity: 0.18 }),
      Plot.ruleY([0], { stroke: GRID, strokeOpacity: 0.6 })
    ]
  });

  return html`<div class="dist-panel"><div class="stats">${stats}</div>${plot}</div>`;
}
```

```
viewof t_nu = Inputs.range([1, 40], {value: 5, step: 1, label: md`${tex`\nu`}`})
```

i Note

Use the tabs to switch distributions. Adjust the sliders to see how the PMF/PDF changes.

Question Bank

Under Construction...